JAVA · IMPORTANT QUESTIONS & IMPORTANT CODE

Java Imp Questions — Tomcat, Servlet, JDBC & Applet

Windows par Tomcat setup → Hello World → web.xml / context.xml → Filters → Forward / Include / Redirect → Cookie & Session → Register-Login-Logout → MySQL JDBC CRUD → Applet.
Har program ka poora code, folder kahan rakhna hai, aur kaise run karna hai.

1 · Tomcat Setup
2 · Config Files
3 · Dispatch & Filters
4 · Cookie & Session
5 · JDBC & CRUD
6 · Applet

Windows 10/11 · JDK 17 · Tomcat 10.1 · MySQL 8 · Code real project files se liya gaya hai

Table of Contents

How to use this book ... 3
- Folders jo hum banayenge (Windows) ... 3

Big picture: ek request ka safar ... 4

Tomcat Setup & Hello World ... 5
- 1.1 Tomcat kya hai? ... 5
- 1.2 Windows par step-by-step install ... 6
- 1.3 Tomcat ka folder structure ... 7
- 1.4 Ek web application ka structure ... 8
- 1.5 Pehla Hello World servlet ... 8
- 1.6 Servlet lifecycle ... 10
- 1.7 URL ka matlab ... 10
- 1.8 Deploy ke tareeke & WAR file ... 11

web.xml, context.xml & Config Files ... 12
- 2.1 Kaunsi config file kahan rakhi hai? ... 12
- 2.2 web.xml (deployment descriptor) ... 12
- 2.3 init-param vs context-param ... 14
- 2.4 context.xml ... 15
- 2.5 server.xml ... 16
- 2.6 tomcat-users.xml (Manager app ke liye) ... 16
- 2.7 Listener (app start / stop events) ... 17
- 2.8 Logs — error kahan dekhein? ... 17

Forward, Include, Redirect & Filters ... 19
- 3.1 forward vs include vs sendRedirect ... 19
- 3.2 Filters ... 23
- 3.3 Filter mapping ... 26
- 3.4 Servlet URL patterns ... 27

Cookie, Session & Register / Login / Logout ... 28
- 4.1 Session tracking kyon chahiye? ... 28
- 4.2 Cookie management ... 28
- 4.3 HttpSession management ... 30
- 4.4 Register + Login + Logout app ... 32

JDBC & MySQL CRUD ... 40
- 5.1 JDBC kya hai? ... 40
- 5.2 MySQL + driver setup (Windows) ... 41
- 5.3 Pehla JDBC program (7 steps) ... 42
- 5.4 CRUD — console program ... 44
- 5.5 Transaction, Batch, Stored procedure ... 46
- 5.6 Servlet me database (MVC style) ... 48
- 5.7 Connection pool (context.xml + JNDI) ... 52

Applet Programs ... 54
- 6.1 Applet kya hai? ... 54
- 6.2 Programs ... 55
- 6.3 Kaise chalayein ... 58
- 6.4 Applet restrictions & comparison ... 59

Troubleshooting — error aaye to ... 60

Quick comparison sheet ... 61

Viva — short Q & A ... 62

Command cheat-sheet (Windows) ... 63

Revision checklist ... 64

How to use this book

⭐ Exam tip — Order me padho

1. **Chapter 1** follow karke Tomcat install karo aur Hello World chalao — baaki sab isi par bana hai.
2. Phir ek project folder `C:\myapp-dev` banao (neeche layout) aur har chapter ki files wahan rakho.
3. **build.bat** chalao → code compile + copy hota hai → Tomcat start → browser me test.

Folders jo hum banayenge (Windows)

Folder	Kaam
`C:\tomcat`	Tomcat ka unzip kiya hua folder. Isi ko **CATALINA_HOME** kehte hain (andar bin, conf, lib, webapps...)
`C:\myapp-dev`	Tumhara **development** folder — yahan code likhte ho (src, web, build.bat, schema.sql)
`C:\tomcat\webapps\myapp`	**Deployed** app — build.bat yahan .class aur web files copy karta hai
`C:\jdbc-practice`	Console JDBC programs + mysql-connector-j jar
`C:\applets`	Applet programs

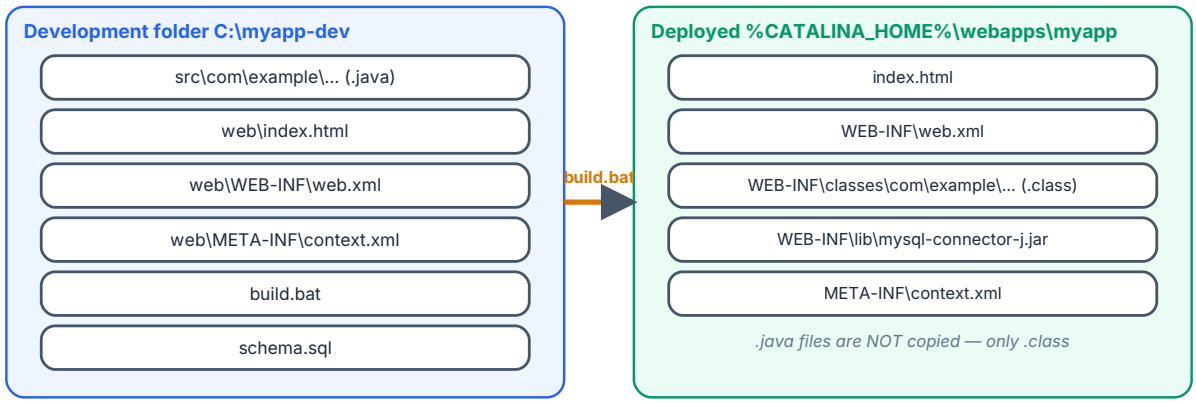
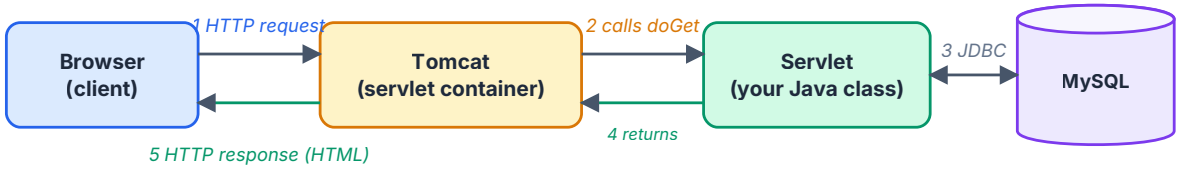


Fig. 1 — Development folder se deployed folder tak

⚠️ Careful — Honest note — kya test hua

Saare Java programs **compile** kiye gaye hain. Servlets ko Tomcat ke asli jar ki jagah API-stubs ke against compile kiya (is machine par internet allowlist hai), isliye servlet pages ka output "expected output" hai — tumhare Tomcat par same chalna chahiye. JDBC programs ko live MySQL par nahi chalaya (yahan DB nahi hai). **Applets** actually chalaye gaye aur screenshot liye gaye.

Big picture: ek request ka safar



The servlet never talks to the browser directly — Tomcat sits in the middle.

Fig. 2 — Browser → Tomcat → Servlet → MySQL → Browser

🧠 **Yaad rakho**

Browser sirf **Tomcat** se baat karta hai. Tomcat tumhare servlet ko call karta hai. Servlet JDBC se MySQL se data laata hai aur HTML bana kar wapas deta hai.

CHAPTER 1

Tomcat Setup & Hello World

Install on Windows | JAVA_HOME / CATALINA_HOME | Folder structure | Hello Servlet | Deploy & WAR

1.1 Tomcat kya hai?

Definition

Apache Tomcat ek open-source **servlet container** (web server) hai jo Java Servlets aur JSP ko chalata hai. Servlet = Java class jo HTTP request leke response (HTML) banati hai.

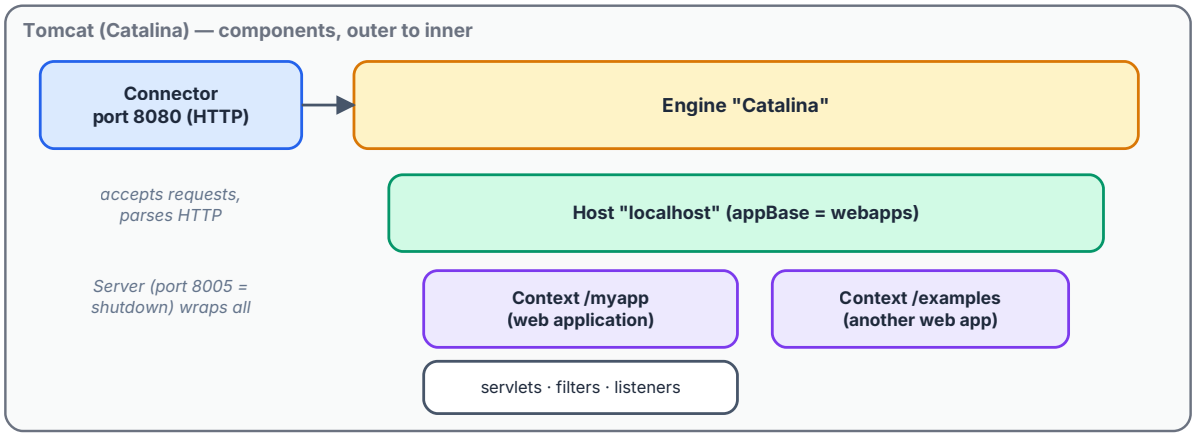


Fig. 3 — Tomcat ke andar: Connector → Engine → Host → Context

Kaunsa Tomcat version lein?

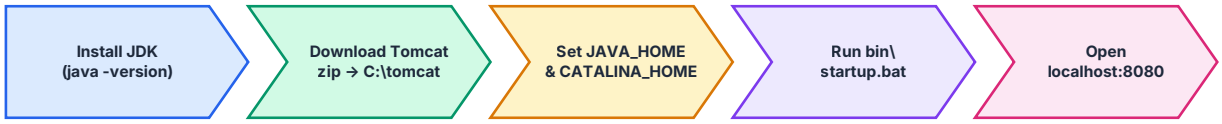
Tomcat	Servlet spec	Package name	Minimum Java
9.0	4.0	javax.servlet.*	Java 8
10.1	6.0	jakarta.servlet.*	Java 11
11.0	6.1	jakarta.servlet.*	Java 17

Code written for javax will NOT run on Tomcat 10+ (change imports to jakarta).

Fig. 4 — Tomcat version vs Servlet spec vs package name

> ⚠️ **Careful — javax vs jakarta — sabse common error**
>
> Tomcat 9 par `import javax.servlet.*` chalta hai. **Tomcat 10+ par** `import jakarta.servlet.*` likhna padta hai. Galat import → **ClassNotFoundException** ya servlet load nahi hota. Ye book **Tomcat 10.1 (jakarta)** use karti hai. Tomcat 9 chahiye to bas imports me `jakarta` ki jagah `javax` likho.

1.2 Windows par step-by-step install



To stop: bin\shutdown.bat

Fig. 5 — Install flow

Step 1 — JDK check

```bash
java -version
javac -version
```

Dono chalne chahiye (JDK 11+ for Tomcat 10.1; **javac** sirf JDK me hota hai, JRE me nahi). Nahi chale to JDK install karo.

Step 2 — Tomcat download & unzip

- `tomcat.apache.org` → Download → Tomcat 10.1 → **64-bit Windows zip**.
- Zip ko **C:\tomcat** me extract karo, taaki `C:\tomcat\bin\startup.bat` exist kare. (Folder me space ya Hindi naam na rakho.)

Step 3 — Environment Variables set karo

1. Start → search **"Edit the system environment variables"** → **Environment Variables...**
2. **New** (System variables) → Name `JAVA_HOME`, Value = JDK folder, e.g. `C:\Program Files\Java\jdk-17` (**bin** tak nahi, sirf jdk folder).
3. **New** → Name `CATALINA_HOME`, Value `C:\tomcat`.
4. OK → OK. **Naya** Command Prompt kholo (purana naye variables nahi dekhta).

```bash
echo %JAVA_HOME%
echo %CATALINA_HOME%
```

Step 4 — Start / Stop

```bash
cd C:\tomcat\bin
startup.bat
REM browser: http://localhost:8080
shutdown.bat
```

Script	Kaam
`startup.bat`	Naye console window me Tomcat start (background)
`catalina.bat run`	Isi window me start — **errors yahin dikhte hain** (debugging ke liye best)
`shutdown.bat`	Tomcat band (port 8005 par shutdown command)
`version.bat`	Tomcat + Java version dikhata hai
`service.bat install`	Windows Service banata hai (auto-start)

Browser me `http://localhost:8080` par Tomcat ka cat-logo page aaye to setup **success**.

Port 8080 busy ho to

```bash
netstat -ano | findstr :8080
taskkill /PID 1234 /F
```

Ya `conf\server.xml` me Connector ka port badal do (section 2.3).

1.3 Tomcat ka folder structure

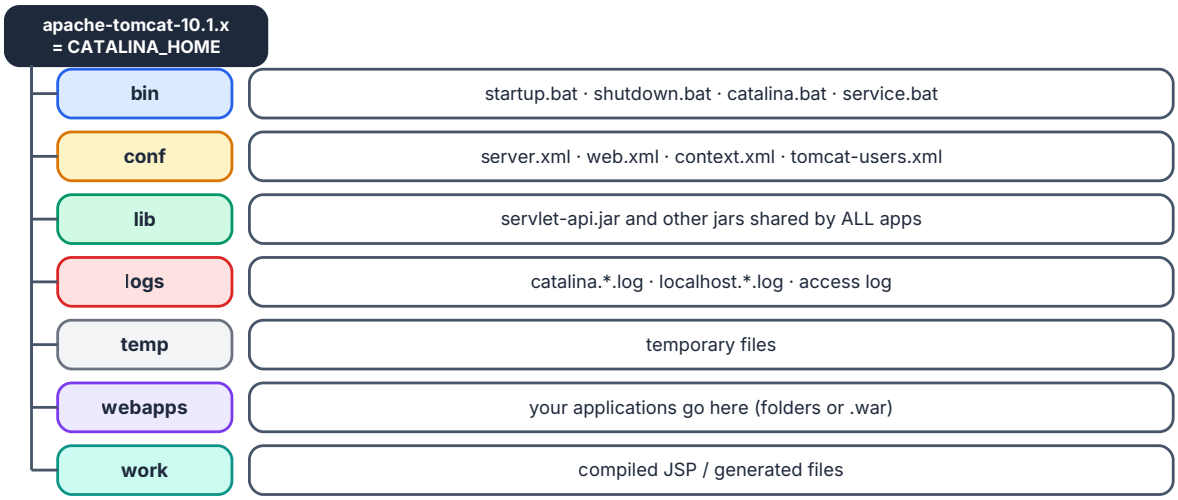


Fig. 6 — C:\tomcat ke andar ke folders

> ⭐ **Exam tip**
>
> Likhna: **bin** (scripts) · **conf** (config files) · **lib** (shared jars, servlet-api.jar) · **logs** (log files) · **temp** · **webapps** (apps deploy hoti hain) · **work** (JSP compile output).

Webapps ke andar	Meaning
`ROOT`	Default app — URL `http://localhost:8080/`
`docs, examples`	Documentation aur sample servlets (`/examples` se seekh sakte ho)
`manager, host-manager`	Admin web apps (tomcat-users.xml me user chahiye)
`myapp` (tumhara)	URL `http://localhost:8080/myapp/`

1.4 Ek web application ka structure

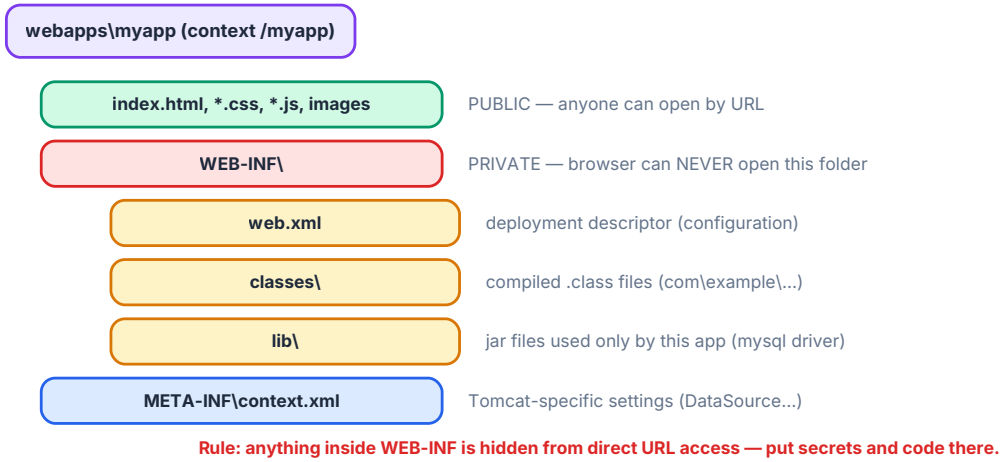


Fig. 7 — Web application ka standard layout

📘 **Definition**

WEB-INF ek special private folder hai: browser se `http://localhost:8080/myapp/WEB-INF/web.xml` khologe to **404** aayega. Isliye classes, jars aur config yahin rehte hain.

🧠 **Yaad rakho**

.java file kabhi webapps me nahi jaati — sirf **.class** jaati hai `WEB-INF\classes` me, package ke folder structure ke saath (`com\example\hello\HelloServlet.class`).

1.5 Pehla Hello World servlet

Step A — folders banao:

```bash
mkdir C:\myapp-dev\src\com\example\hello
mkdir C:\myapp-dev\web\WEB-INF
mkdir C:\myapp-dev\web\META-INF
```

Step B — file: `C:\myapp-dev\src\com\example\hello\HelloServlet.java`

HelloServlet.java — Hello World servlet (@WebServlet("/hello"))

JAVA · 29 LINES

```java
package com.example.hello;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

// URL: http://localhost:8080/myapp/hello
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
```

```
        response.setContentType("text/html;charset=UTF-8");   // 1. tell browser what we send
        PrintWriter out = response.getWriter();               // 2. get the output stream
        String name = request.getParameter("name");           // ?name=Rohit
        if (name == null || name.isBlank()) {
            name = "World";
        }
        out.println("<html><body>");
        out.println("<h1>Hello, " + name + "!</h1>");         // (escape 'name' in real apps)
        out.println("<p>This page is generated by a Servlet.</p>");
        out.println("</body></html>");
    }
}
```

- **@WebServlet("/hello")** — URL mapping annotation; `web.xml` me likhna zaroori nahi.
- **extends HttpServlet** — `doGet()` GET request ke liye, `doPost()` POST ke liye.
- **request.getParameter("name")** — URL ke `?name=Rohit` ki value.
- **response.getWriter()** — browser ko HTML text bhejne ka stream.

Step C — chhota web.xml (Tomcat 10.1 / Servlet 6.0):

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
           https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
         version="6.0">
    <display-name>myapp</display-name>
</web-app>
```

Step D — compile aur deploy. Pura project aage chapters me bada hoga; build.bat usi ke liye hai:

build.bat C:\myapp-dev\build.bat — compile + copy

BAT · 19 LINES

```bat
@echo off
REM ---- Edit this line if Tomcat is somewhere else ----
set CATALINA_HOME=C:\tomcat
set APP=%CATALINA_HOME%\webapps\myapp

if not exist "%APP%\WEB-INF\classes" mkdir "%APP%\WEB-INF\classes"
if not exist "%APP%\WEB-INF\lib" mkdir "%APP%\WEB-INF\lib"

REM 1. make a list of all .java files (works for any number of packages)
dir /s /b src\*.java > sources.txt

REM 2. compile into WEB-INF\classes  (servlet-api.jar comes from Tomcat itself)
javac -encoding UTF-8 -cp "%CATALINA_HOME%\lib\servlet-api.jar" -d "%APP%\WEB-INF\classes" @sources.txt
if errorlevel 1 (echo COMPILE FAILED & exit /b 1)

REM 3. copy html, WEB-INF\web.xml, META-INF\context.xml
xcopy web "%APP%" /E /I /Y > nul

echo Done. Start Tomcat and open http://localhost:8080/myapp/
```

BASH

```bash
cd C:\myapp-dev
build.bat
C:\tomcat\bin\startup.bat
```

Step E — test: browser me `http://localhost:8080/myapp/hello?name=Rohit`

⭐ **Exam tip — Expected output (browser)**

Page par bada heading **Hello, Rohit!** aur neeche line "This page is generated by a Servlet." dikhegi. Bina `?name=` ke heading **Hello, World!** hogi.

1.6 Servlet lifecycle

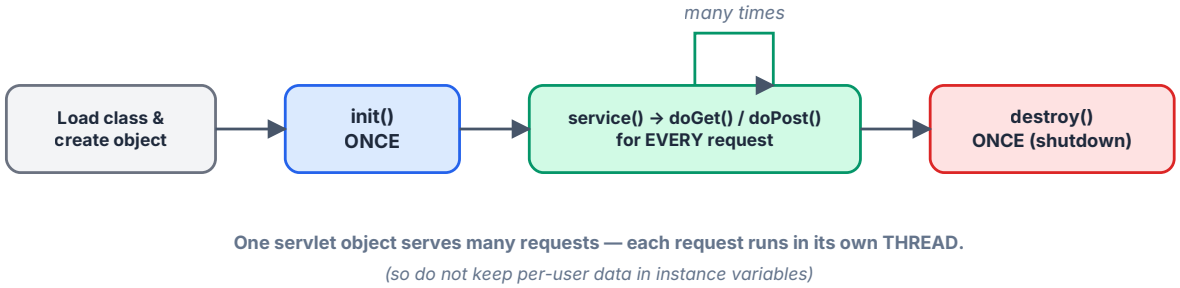


Fig. 8 — Servlet lifecycle

Method	Kab chalta hai
`init()`	Pehli request par (ya startup par agar `load-on-startup`) — sirf **ek baar**
`service()`	Har request par; ye request type dekhke `doGet / doPost` call karta hai
`destroy()`	Tomcat band hone ya app undeploy hone par — ek baar

⚠️ **Careful — Thread safety**

Ek servlet ka **sirf ek object** banta hai, har request alag **thread** me chalti hai. Isliye user-specific data **instance variable** me mat rakho — local variable ya session me rakho.

1.7 URL ka matlab

`http://localhost:8080/myapp/hello?name=Rohit`

http://	localhost	:8080	/myapp	/hello	?name=Rohit
protocol	host	port	context path (app folder)	servlet path (@WebServlet)	query string

```
getContextPath() = /myapp  getServletPath() = /hello
getParameter("name") = Rohit
```

Fig. 9 — URL ke hisse

Pattern	Type	Matches	Priority
/hello	exact	only /hello	1 (highest)
/students/*	path	/students/1, /students/new	2
*.do	extension	login.do, save.do	3
/	default	anything not matched	4 (lowest)

Fig. 10 — url-pattern ke types

1.8 Deploy ke tareeke & WAR file

Tareeka	Kaise
Exploded folder	`webapps\myapp\` folder banao (hum yehi use karte hain)
WAR file	`myapp.war` ko `webapps` me copy → Tomcat khud unzip karta hai
Manager app	`/manager/html` se upload (tomcat-users.xml me `manager-gui` role chahiye)
ROOT app	Folder/WAR ka naam `ROOT` → URL `/`

```bash
cd C:\tomcat\webapps\myapp
jar cvf ..\myapp.war *
```

Context path = folder (ya war) ka naam. `myapp.war` → `/myapp`.

Change kab apply hota hai?

Badlav	Kya karna hai
`web.xml` / HTML / JSP	Aksar auto-reload; nahi to app reload
Java class recompile	**Tomcat restart** (ya context.xml me `reloadable="true"` — sirf development me)
`conf\server.xml`	Tomcat restart zaroori

❓ Likely exam questions — Chapter 1 — Likely exam questions

1. Tomcat kya hai? Uska architecture (Connector, Engine, Host, Context) samjhao.
2. Windows par Tomcat install karne ke steps likho. JAVA_HOME aur CATALINA_HOME kya hain?
3. Tomcat ke directories (bin, conf, lib, logs, webapps, work) ka kaam batao.
4. Web application ka standard folder structure draw karo. WEB-INF kyon private hai?
5. Hello World servlet likho aur deploy karne ka tareeka samjhao.
6. Servlet lifecycle methods samjhao. Servlet thread-safe kyon nahi hota?
7. javax aur jakarta me kya difference hai?
8. WAR file kya hai? Kaise banate hain?

CHAPTER 2

web.xml, context.xml & Config Files

Which file where | web.xml tags | context.xml | server.xml | tomcat-users.xml | init-param vs context-param | Logs

2.1 Kaunsi config file kahan rakhi hai?

File	Location	Scope	Purpose
server.xml	conf\	whole server	ports, Connector, Host, appBase
web.xml (global)	conf\	all apps	default servlet, MIME types, defaults
context.xml (global)	conf\	all apps	settings shared by every context
tomcat-users.xml	conf\	server	users & roles (Manager app)
web.xml (yours)	WEB-INF\	one app	servlets, filters, params, error pages
context.xml (yours)	META-INF\	one app	DataSource, env values, reload

Fig. 11 — Tomcat ki config files — location, scope, kaam

🧠 **Yaad rakho**

Same naam ki file **do jagah** hoti hai: `conf\web.xml` / `conf\context.xml` = **poore Tomcat** ke liye. `WEB-INF\web.xml` / `META-INF\context.xml` = **sirf tumhari app** ke liye. App wali file global ko override karti hai.

2.2 web.xml (deployment descriptor)

📘 **Definition**

web.xml web application ka configuration file hai (`WEB-INF\web.xml`). Servlet, filter, listener, parameters, session timeout, error pages, welcome files yahin likhte hain. Servlet 3.0+ me **@WebServlet / @WebFilter** annotations se bina web.xml ke bhi kaam chalta hai.

Poori web.xml (humare project ki)

web.xml C:\myapp-dev\web\WEB-INF\web.xml

XML · 85 LINES

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
                             https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
         version="6.0">

    <display-name>My College App</display-name>
```

```
<!-- 1. context-param: value visible to ALL servlets (ServletContext) -->
<context-param>
    <param-name>college</param-name>
    <param-value>ABC Engineering College</param-value>
</context-param>

<!-- 2. filters: declared here, then mapped. Order of filter-mapping = order of execution -->
<filter>
    <filter-name>EncodingFilter</filter-name>
    <filter-class>com.example.filter.EncodingFilter</filter-class>
    <init-param>
        <param-name>encoding</param-name>
        <param-value>UTF-8</param-value>
    </init-param>
</filter>
<filter-mapping>
    <filter-name>EncodingFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

<filter>
    <filter-name>LogFilter</filter-name>
    <filter-class>com.example.filter.LogFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>LogFilter</filter-name>
    <url-pattern>/*</url-pattern>
    <dispatcher>REQUEST</dispatcher>
    <dispatcher>FORWARD</dispatcher>
</filter-mapping>

<!-- 3. servlet declared in XML (instead of @WebServlet) with its own init-param -->
<servlet>
    <servlet-name>InitDemo</servlet-name>
    <servlet-class>com.example.hello.InitDemoServlet</servlet-class>
    <init-param>
        <param-name>author</param-name>
        <param-value>Rohit</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>InitDemo</servlet-name>
    <url-pattern>/init-demo</url-pattern>
</servlet-mapping>

<!-- 4. session settings -->
<session-config>
    <session-timeout>30</session-timeout>        <!-- minutes -->
    <cookie-config>
        <http-only>true</http-only>
    </cookie-config>
</session-config>

<!-- 5. first page shown for http://localhost:8080/myapp/ -->
<welcome-file-list>
    <welcome-file>index.html</welcome-file>
</welcome-file-list>

<!-- 6. custom error pages -->
<error-page>
    <error-code>404</error-code>
    <location>/error404.html</location>
</error-page>
<error-page>
    <exception-type>java.lang.Exception</exception-type>
    <location>/error500.html</location>
</error-page>

<!-- 7. only needed if you use the pooled DataSource from context.xml (JndiDB) -->
<resource-ref>
    <res-ref-name>jdbc/college</res-ref-name>
```

```
        <res-type>javax.sql.DataSource</res-type>
        <res-auth>Container</res-auth>
    </resource-ref>
</web-app>
```

Tags ka summary

Tag	Kaam
`<web-app version="6.0">`	Root tag; Tomcat 10.1 = Servlet 6.0 (Tomcat 9 = `4.0`, xmlns `java.sun.com` / `xmlns.jcp.org`)
`<context-param>`	Poori app ke liye value (**ServletContext**)
`<servlet> + <servlet-mapping>`	Servlet class declare + URL se jodna (name dono me same)
`<init-param>`	Ek servlet/filter ke liye value (**ServletConfig**)
`<load-on-startup>`	Startup par hi load (1 = pehle). Warna pehli request par
`<filter> + <filter-mapping>`	Filter declare + map (chapter 3)
`<listener>`	Event listener register
`<session-config>`	Session timeout (minutes), cookie settings
`<welcome-file-list>`	`/myapp/` par kaunsi file khule
`<error-page>`	404 / 500 / exception par custom page
`<resource-ref>`	context.xml ke DataSource ko app me declare

⭐ **Exam tip — Annotation vs web.xml**

Annotation — chhota, code ke paas (`@WebServlet("/hello")`). **web.xml** — code badle bina config badal sakte ho; init-param, load-on-startup, filter **order** yahin control hota hai. Dono ek saath chal sakte hain; same naam par **web.xml jeetta hai**.

2.3 init-param vs context-param

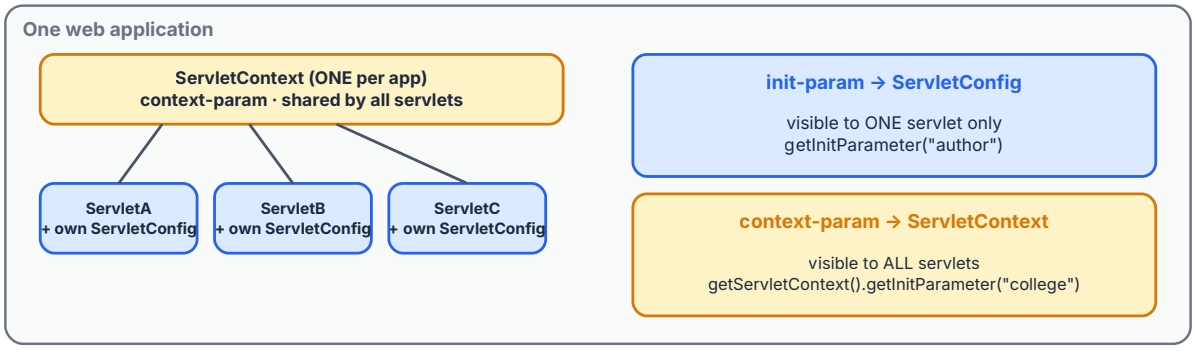


Fig. 12 — ServletConfig vs ServletContext

InitDemoServlet.java — dono parameters padhne ka code

```
JAVA · 33 LINES
package com.example.hello;
```

```java
import jakarta.servlet.ServletContext;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

// Mapped in web.xml (not by annotation): shows <init-param> and <context-param>
public class InitDemoServlet extends HttpServlet {

    private String author;                      // from <init-param>  (this servlet only)

    @Override
    public void init() throws ServletException {  // runs ONCE, when the servlet is created
        author = getInitParameter("author");
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/plain;charset=UTF-8");
        PrintWriter out = response.getWriter();
        ServletContext ctx = getServletContext();
        out.println("init-param    author  = " + author);
        out.println("context-param college = " + ctx.getInitParameter("college"));
        out.println("context path          = " + request.getContextPath());
        out.println("servlet path          = " + request.getServletPath());
        out.println("real path of /        = " + ctx.getRealPath("/"));
    }
}
```

	init-param	context-param
Object	`ServletConfig`	`ServletContext`
Scope	ek servlet / filter	poori application
Read	`getInitParameter("author")`	`getServletContext().getInitParameter("college")`
Tag location	<servlet> ke andar	<web-app> ke andar

Expected output (`/myapp/init-demo`): author = Rohit, college = ABC Engineering College.

2.4 context.xml

📘 **Definition**

context.xml Tomcat-specific file hai (`META-INF\context.xml`). Isme DataSource (connection pool), environment values, `reloadable` jaisi Tomcat settings hoti hain. **Servlet spec ka hissa nahi** — sirf Tomcat samajhta hai.

context.xml C:\myapp-dev\web\META-INF\context.xml

XML · 18 LINES

```xml
<?xml version="1.0" encoding="UTF-8"?>
<Context reloadable="false">

    <!-- Connection POOL managed by Tomcat. Needs mysql-connector-j.jar in %CATALINA_HOME%\lib -->
    <Resource name="jdbc/college"
              auth="Container"
              type="javax.sql.DataSource"
              driverClassName="com.mysql.cj.jdbc.Driver"
              url="jdbc:mysql://localhost:3306/college?serverTimezone=UTC"
              username="root"
              password="root123"
```

```
            maxTotal="20"
            maxIdle="5"
            maxWaitMillis="10000"/>

    <!-- A value any servlet can read through JNDI (optional example) -->
    <Environment name="maxStudents" value="500" type="java.lang.Integer" override="false"/>
</Context>
```

Attribute	Meaning
`reloadable`	true = classes badalne par auto-reload (sirf development; slow)
`Resource name`	JNDI naam — code me `java:comp/env/jdbc/college`
`maxTotal / maxIdle`	Pool me max connections / idle connections
`maxWaitMillis`	Connection na mile to kitni der wait (ms)
`Environment`	Config value jo JNDI se padhi ja sakti hai

Alternative: context.xml ko `conf\Catalina\localhost\myapp.xml` naam se bhi rakh sakte ho (webapp ke bahar).

2.5 server.xml

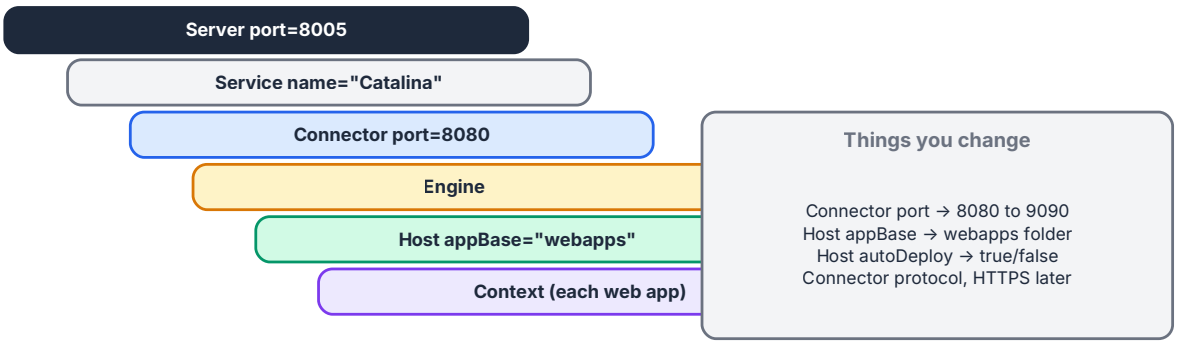


Fig. 13 — server.xml ka hierarchy

```xml
<Connector port="8080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443" />
```

Port **8080 → 9090** karne ke liye `port="9090"` likho aur Tomcat restart karo; phir URL `http://localhost:9090/myapp`.

2.6 tomcat-users.xml (Manager app ke liye)

```xml
<tomcat-users>
  <role rolename="manager-gui"/>
  <user username="admin" password="admin123" roles="manager-gui"/>
</tomcat-users>
```

Save karke restart → `http://localhost:8080/manager/html`. Manager app default me **sirf localhost** se khulta hai (`webapps\manager\META-INF\context.xml` me RemoteAddrValve).

2.7 Listener (app start / stop events)

AppListener.java — @WebListener — start/stop aur session counter

JAVA · 36 LINES

```java
package com.example.listener;

import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.annotation.WebListener;
import jakarta.servlet.http.HttpSessionEvent;
import jakarta.servlet.http.HttpSessionListener;
import java.util.concurrent.atomic.AtomicInteger;

/** Runs once at application start/stop, and whenever a session is created/destroyed. */
@WebListener
public class AppListener implements ServletContextListener, HttpSessionListener {

    public static final AtomicInteger ACTIVE_SESSIONS = new AtomicInteger();

    @Override
    public void contextInitialized(ServletContextEvent e) {
        e.getServletContext().setAttribute("appName", "College Portal");
        System.out.println("[AppListener] application started");
    }

    @Override
    public void contextDestroyed(ServletContextEvent e) {
        System.out.println("[AppListener] application stopped");
    }

    @Override
    public void sessionCreated(HttpSessionEvent e) {
        ACTIVE_SESSIONS.incrementAndGet();
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent e) {
        ACTIVE_SESSIONS.decrementAndGet();
    }
}
```

Listener	Fires when	Use
ServletContextListener	app starts / stops	load config, open pool
HttpSessionListener	session created / destroyed	count online users
ServletRequestListener	request starts / ends	timing, logging

Fig. 14 — Listener types

2.8 Logs — error kahan dekhein?

File (C:\tomcat\logs)	Kya milta hai
`catalina.YYYY-MM-DD.log`	Tomcat start/stop, deploy errors
`localhost.YYYY-MM-DD.log`	Tumhari app ke exceptions / stack trace
`localhost_access_log…txt`	Har request ki line (URL, status code)

Console me turant dekhne ke liye `catalina.bat run` use karo. Servlet me `getServletContext().log("msg")` ya `System.out.println` (→ `catalina.out` /console) likh sakte ho.

❓ Likely exam questions — Chapter 2 — Likely exam questions

1. web.xml kya hai? Important tags explain karo.
2. web.xml aur context.xml me difference. Dono kahan rakhte hain?
3. init-param aur context-param me difference (ServletConfig vs ServletContext).
4. @WebServlet annotation aur web.xml mapping dono ka example do.
5. context.xml me DataSource kaise configure karte hain?
6. server.xml me port kaise badalte hain?
7. load-on-startup kya hai? Welcome file aur error-page kaise set karte hain?
8. Error ke liye Tomcat ke kaunse log files dekhte hain?

CHAPTER 3

Forward, Include, Redirect & Filters

RequestDispatcher | forward | include | sendRedirect | Filter chain | Filter mapping | Encoding / Log / Auth filter

3.1 forward vs include vs sendRedirect

`RequestDispatcher .forward(req,res)`	`RequestDispatcher .include(req,res)`	`response .sendRedirect(url)`
Server-side, same request	Server-side, output merged	Browser makes NEW request (302)
URL does NOT change	URL does NOT change	URL CHANGES
request attributes kept	included page cannot set headers	request attributes LOST

Forward / include stay inside the server; redirect goes back to the browser first.

Fig. 15 — Teeno methods ka comparison

Definition

RequestDispatcher = ek object jo request ko server ke andar kisi dusre resource (servlet / JSP / HTML) ko bhejta hai. Milta hai: `request.getRequestDispatcher("/target")` ya `getServletContext().getRequestDispatcher("/target")`.

forward()

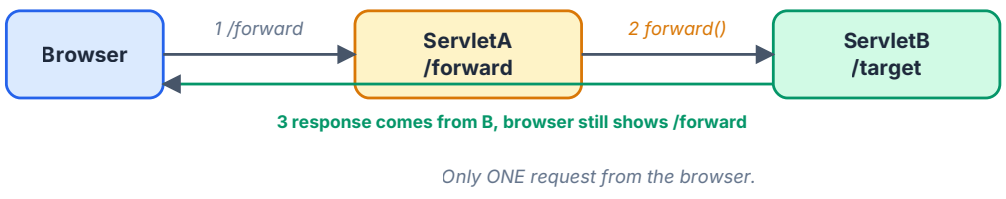


Fig. 16 — forward() — ek hi request

ForwardDemoServlet.java — /forward-demo → forward to /target

JAVA · 23 LINES

```java
package com.example.web;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
```

```java
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

/** forward: server-side jump. URL in browser stays /forward-demo. Request attributes survive. */
@WebServlet("/forward-demo")
public class ForwardDemoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.setAttribute("msg", "Data set in ForwardDemoServlet");   // travels with the request
        RequestDispatcher rd = request.getRequestDispatcher("/target");
        rd.forward(request, response);
        // Do NOT write anything to 'response' after forward()
    }
}
```

TargetServlet.java — /target — attribute padhta hai

JAVA · 22 LINES

```java
package com.example.web;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

/** Receives the forward / redirect. */
@WebServlet("/target")
public class TargetServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/plain;charset=UTF-8");
        Object msg = request.getAttribute("msg");
        response.getWriter().println("URL shown in browser : " + request.getRequestURI());
        response.getWriter().println("request attribute msg: " + (msg == null ? "(null - attribute was
lost)" : msg));
    }
}
```

- Browser URL **/forward-demo** hi dikhata hai, par page **/target** ka hai.
- `request.setAttribute(...)` target ko mil jaata hai (same request object).
- forward ke baad **koi aur output mat likho** — response commit ho chuka hota hai.

include()

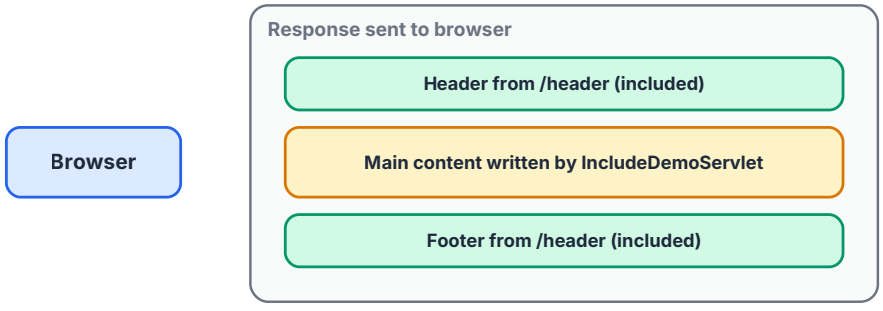


Fig. 17 — include() — output merge

IncludeDemoServlet.java /include-demo — /header ko include karta hai

JAVA · 23 LINES

```java
package com.example.web;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

/** include: output of another resource is inserted INTO this response; control comes back. */
@WebServlet("/include-demo")
public class IncludeDemoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().println("<html><body>");
        request.getRequestDispatcher("/header").include(request, response);   // part 1 from HeaderServlet
        response.getWriter().println("<p>This text is from IncludeDemoServlet.</p>");   // we continue after
include
        response.getWriter().println("</body></html>");
    }
}
```

HeaderServlet.java /header — reusable header/footer

JAVA · 29 LINES

```java
package com.example.web;

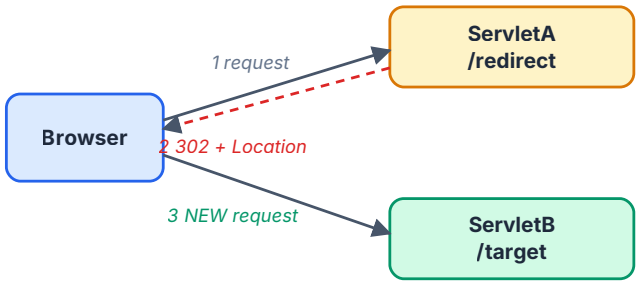
import com.example.model.User;
import com.example.util.Html;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

/** A reusable page fragment (menu bar). Other servlets INCLUDE it. */
@WebServlet("/header")
public class HeaderServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String ctx = request.getContextPath();
        HttpSession session = request.getSession(false);
        User user = session == null ? null : (User) session.getAttribute("user");
        String who = user == null ? "Guest" : Html.esc(user.getName());
        response.getWriter().println("<div style='background:#eef;padding:8px'>"
            + "<a href='" + ctx + "/dashboard'>Dashboard</a> | "
            + "<a href='" + ctx + "/students'>Students</a> | "
            + "<a href='" + ctx + "/logout'>Logout</a>   (" + who + ")</div>");
    }
}
```

Included servlet **headers ya status code set nahi kar sakta** (ignore ho jaate hain). Use: common header/footer/ menu.

sendRedirect()



Browser URL becomes /target
Two requests in total
req.setAttribute() is lost
Use for: after POST (PRG), after login, external sites

Fig. 18 — sendRedirect — browser naya request karta hai

RedirectDemoServlet.java /redirect-demo

JAVA · 20 LINES

```java
package com.example.web;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;

/** sendRedirect: tells the BROWSER to make a brand-new request. URL changes; request attributes are lost. */
@WebServlet("/redirect-demo")
public class RedirectDemoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.setAttribute("msg", "This will NOT reach /target after a redirect");
        response.sendRedirect(request.getContextPath() + "/target");   // 302 + Location header
    }
}
```

Comparison table (exam favourite)

Point	forward	include	sendRedirect
Kaun karta hai	Server	Server	Browser (302)
Requests	1	1	2
URL change	Nahi	Nahi	Haan
Request attributes	Rehte hain	Rehte hain	Khatam
Output	Sirf target ka	Dono ka mix	Naya page
Dusri site par?	Nahi (same app)	Nahi	Haan
Kab use	Controller → view	Header/footer	POST ke baad, login ke baad

🧠 **Yaad rakho**

Form submit (POST) ke baad **redirect** karo — warna page refresh karne par form dobara submit hota hai. Ise **Post-Redirect-Get** kehte hain (StudentServlet me use kiya hai).

Aur useful methods

Method	Kaam
`request.getParameter("x")`	Form / query value (String)
`request.setAttribute / getAttribute`	Servlets ke beech data pass (request scope)
`request.getContextPath()`	`/myapp` — redirect URL banane me
`response.setContentType`	text/html, application/json...
`response.sendError(404, "msg")`	Error status bhejna
`response.setHeader(...)`	Custom header
`response.encodeURL(url)`	URL rewriting (cookie band ho to session id add)

3.2 Filters

Definition

Filter ek Java class hai jo request servlet tak pahunchne **se pehle** aur response browser tak jaane **se pehle** beech me chalti hai. Use: encoding set karna, logging, authentication check, compression, CORS.

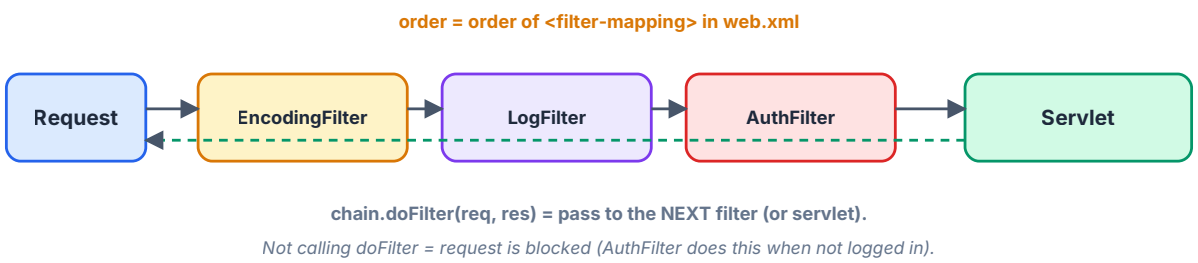


Fig. 19 — Filter chain — order web.xml ke order me

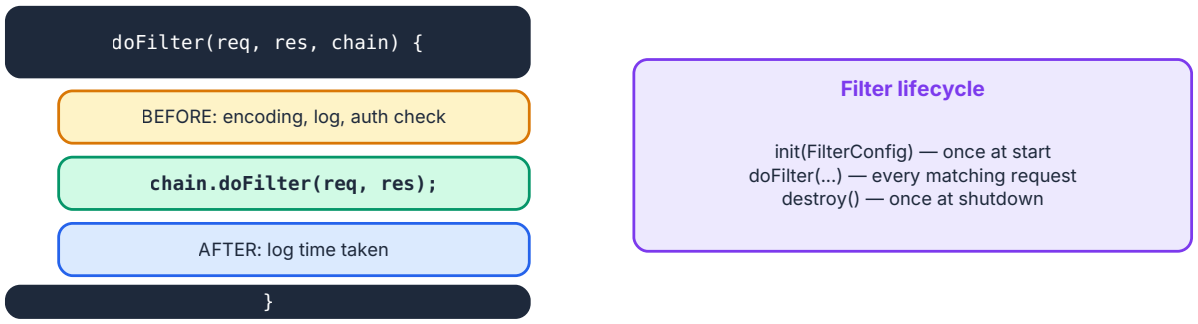


Fig. 20 — doFilter() ka structure aur lifecycle

EncodingFilter (init-param ke saath)

EncodingFilter.java — har request par UTF-8 set

JAVA · 28 LINES

```java
package com.example.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
```

```java
import jakarta.servlet.FilterConfig;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import java.io.IOException;

/** Sets UTF-8 on every request so Hindi / special characters in forms are not garbled. */
public class EncodingFilter implements Filter {
    private String encoding = "UTF-8";

    @Override
    public void init(FilterConfig config) throws ServletException {
        String e = config.getInitParameter("encoding");     // <init-param> in web.xml
        if (e != null) encoding = e;
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        request.setCharacterEncoding(encoding);             // BEFORE the servlet reads parameters
        response.setCharacterEncoding(encoding);
        chain.doFilter(request, response);                  // pass control to the next filter / servlet
    }
}
```

LogFilter (time naapna)

LogFilter.java — URL + time taken print

JAVA · 25 LINES

```java
package com.example.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import java.io.IOException;

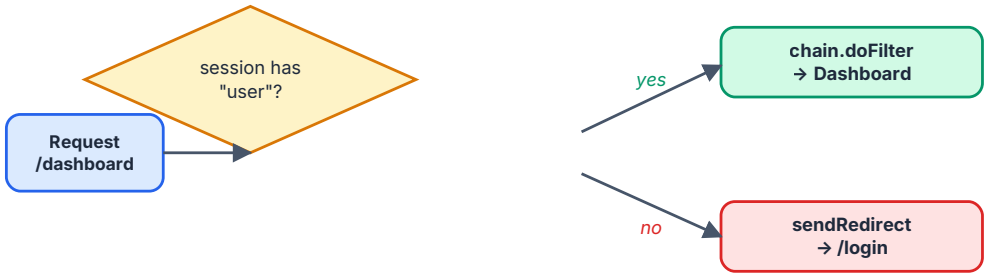
/** Logs "METHOD /uri  (time ms)" for every request. Mapped to /* in web.xml. */
public class LogFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        long start = System.currentTimeMillis();

        chain.doFilter(request, response);                  // ---- servlet runs here ----

        long took = System.currentTimeMillis() - start;     // code AFTER chain = response path
        System.out.println(req.getMethod() + " " + req.getRequestURI() + "  (" + took + " ms)");
    }
}
```

AuthFilter (login check — @WebFilter)



One filter protects many pages — no check code inside each servlet.

Fig. 21 — AuthFilter ka decision

AuthFilter.java — sirf logged-in user aage jaa sakta hai

JAVA · 34 LINES

```java
package com.example.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

/** Protects private pages: no logged-in user in the session -> redirect to /login. */
@WebFilter(urlPatterns = {"/dashboard", "/students", "/students/*"})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);          // false = do NOT create a new one
        boolean loggedIn = session != null && session.getAttribute("user") != null;

        if (loggedIn) {
            res.setHeader("Cache-Control", "no-store");       // back button after logout shows nothing
            chain.doFilter(request, response);                // allowed -> continue
        } else {
            res.sendRedirect(req.getContextPath() + "/login");  // blocked -> servlet never runs
        }
    }
}
```

3.3 Filter mapping

Mapping type	web.xml tag	Example	Filters...
URL pattern	<url-pattern>	/* or /students/*	requests matching URL
Servlet name	<servlet-name>	InitDemo	one specific servlet
Dispatcher	<dispatcher>	REQUEST / FORWARD / INCLUDE / ERROR	internal forwards too

Default dispatcher = REQUEST only. A filter does NOT run on forward() unless <dispatcher>FORWARD</dispatcher> is added.

Fig. 22 — Filter mapping ke 3 tareeke

web.xml ka mapping (humare project se):

```xml
<filter>
    <filter-name>LogFilter</filter-name>
    <filter-class>com.example.filter.LogFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>LogFilter</filter-name>
    <url-pattern>/*</url-pattern>
    <dispatcher>REQUEST</dispatcher>
    <dispatcher>FORWARD</dispatcher>
</filter-mapping>

<!-- servlet-name se mapping -->
<filter-mapping>
    <filter-name>LogFilter</filter-name>
    <servlet-name>InitDemo</servlet-name>
</filter-mapping>
```

Annotation se:

```java
@WebFilter(urlPatterns = {"/dashboard", "/students", "/students/*"})
public class AuthFilter implements Filter { ... }
```

Point	Detail
Execution order	web.xml me **<filter-mapping> ka jo order, wahi execution order**. Annotation ka order **unspecified** hai — order chahiye to web.xml use karo.
Dispatcher	Default sirf REQUEST. forward/include par filter chalane ke liye FORWARD / INCLUDE likho.
Block karna	`chain.doFilter()` call na karo → request aage nahi jaati.
Wrapper	Request/Response ko `HttpServletRequestWrapper` se modify bhi kar sakte hain (advanced).

⚠ Careful — Common galti

Filter ke andar `chain.doFilter(req, res)` bhoolna → page blank aata hai. AuthFilter me `/login` ko map mat karo, warna infinite redirect loop ban jaata hai.

3.4 Servlet URL patterns

Pattern	Type	Matches	Priority
/hello	exact	only /hello	1 (highest)
/students/*	path	/students/1, /students/new	2
*.do	extension	login.do, save.do	3
/	default	anything not matched	4 (lowest)

Fig. 23 — url-pattern ke types aur priority

❓ Likely exam questions — Chapter 3 — Likely exam questions

1. forward, include aur sendRedirect me difference likho (table + example).
2. RequestDispatcher kya hai? Code se forward aur include dikhao.
3. Filter kya hai? Lifecycle methods aur use cases batao.
4. Filter chain kaise kaam karti hai? Order kaise decide hota hai?
5. Filter mapping ke tareeke (url-pattern, servlet-name, dispatcher) example ke saath.
6. Authentication filter likho jo bina login ke /dashboard block kare.
7. Annotation aur web.xml se filter/servlet map karna.
8. Post-Redirect-Get pattern kya hai aur kyon zaroori hai?

CHAPTER 4

Cookie, Session & Register / Login / Logout

HTTP stateless | Cookie | HttpSession | URL rewriting | Register | Login | Logout | Security

4.1 Session tracking kyon chahiye?

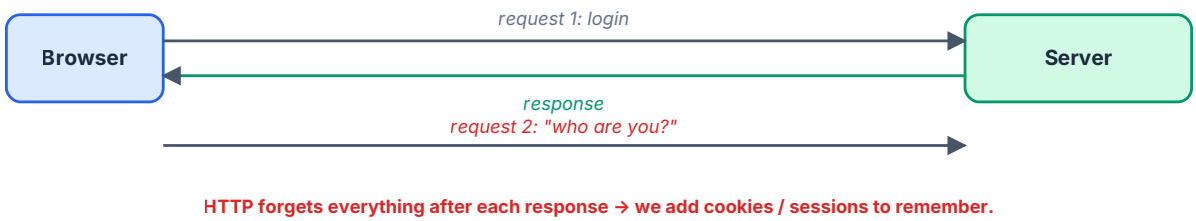


Fig. 24 — HTTP stateless hai

Technique	Where data lives	Pros	Cons
Cookie	browser	survives restart	user can disable / edit
Hidden field	inside HTML form	simple	only form submits
URL rewriting	in the URL (;jsessionid=)	works w/o cookies	ugly, leaks id
HttpSession	SERVER (id in cookie)	secure, any object	uses server memory

Fig. 25 — Session tracking ke 4 tareeke

4.2 Cookie management

📘 **Definition**

Cookie = chhota name-value data jo server browser ko bhejta hai (`Set-Cookie` header); browser har agle request me wapas bhejta hai (`Cookie` header). Size ~4 KB, user dekh/badal sakta hai — isliye password ya sensitive data kabhi mat rakho.

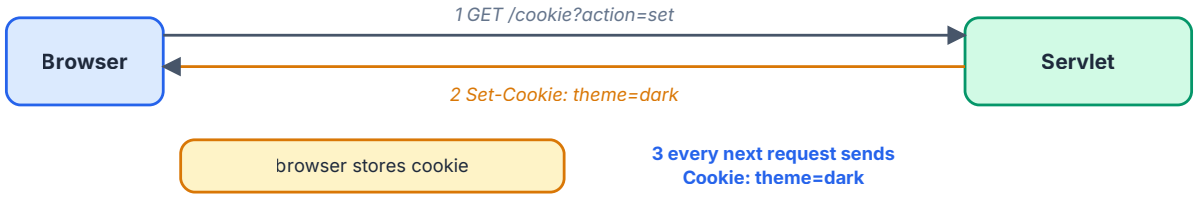


Fig. 26 — Cookie ka flow

setMaxAge(n)	Meaning
n > 0	lives n SECONDS (86400 = 1 day)
n = 0	DELETE cookie now
n < 0 (default)	until browser closes (session cookie)

Fig. 27 — setMaxAge ka matlab

CookieDemoServlet.java — /cookie?action=set | read | delete

JAVA · 54 LINES

```java
package com.example.web;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

/** /cookie?action=set | read | delete */
@WebServlet("/cookie")
public class CookieDemoServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/plain;charset=UTF-8");
        PrintWriter out = response.getWriter();
        String action = request.getParameter("action");
        if (action == null) action = "read";

        switch (action) {
            case "set": {
                Cookie c = new Cookie("theme", "dark");
                c.setMaxAge(60 * 60 * 24);        // seconds; omit/-1 = lives until browser closes
                c.setPath("/");
                c.setHttpOnly(true);
                response.addCookie(c);            // must be called BEFORE the body is committed
                out.println("Cookie 'theme=dark' sent for 1 day.");
                break;
            }
            case "delete": {
                Cookie c = new Cookie("theme", "");
                c.setMaxAge(0);                   // 0 = delete
                c.setPath("/");                   // same path as when it was created
```

```
                response.addCookie(c);
                out.println("Cookie 'theme' deleted.");
                break;
            }
            default: {
                Cookie[] all = request.getCookies();
                if (all == null) {
                    out.println("No cookies received.");
                } else {
                    for (Cookie c : all) {
                        out.println(c.getName() + " = " + c.getValue());
                    }
                }
            }
        }
    }
}
```

```java
Cookie c = new Cookie("user", "Rohit");   // create
c.setMaxAge(60 * 60 * 24);                // 1 day (seconds)
c.setPath("/myapp");                      // kis path par bhejna hai
c.setHttpOnly(true);                      // JavaScript se hide
response.addCookie(c);                    // browser ko bhejo

Cookie[] all = request.getCookies();      // read — null ho sakta hai!
```

⚠️ Careful — Cookie delete karna

Same **name aur path** wala cookie banao, `setMaxAge(0)` karo aur `addCookie` karo. Path alag hua to delete nahi hoga.

Test: `/cookie?action=set` → `/cookie?action=read` (value dikhegi) → `action=delete` → read (khali). Browser me F12 → Application → Cookies me bhi dikhega.

4.3 HttpSession management

📘 Definition

HttpSession me data **server** par rehta hai; browser ko sirf random **JSESSIONID** cookie milti hai. Har user ka alag session object.

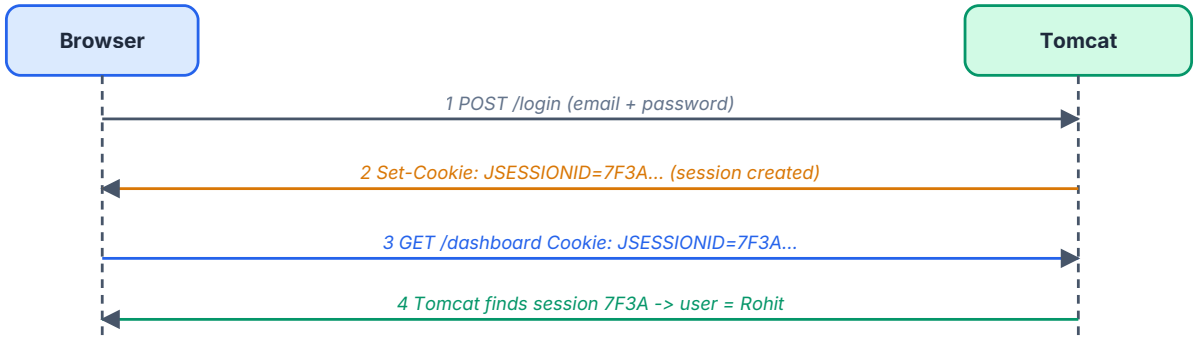


Fig. 28 — Session ka flow

Method	Does
request.getSession()	get session (CREATE if none)
request.getSession(false)	get session or null — use in logout / auth check
setAttribute / getAttribute	store / read an object
invalidate()	destroy session (logout)
setMaxInactiveInterval(s)	idle timeout in seconds

Fig. 29 — HttpSession ke important methods

CounterServlet.java /counter — session me visit count

JAVA · 29 LINES

```java
package com.example.web;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

/** Simplest session example: a visit counter that belongs to ONE browser session. */
@WebServlet("/counter")
public class CounterServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession();              // creates one if needed
        Integer count = (Integer) session.getAttribute("count");
        count = (count == null) ? 1 : count + 1;
        session.setAttribute("count", count);

        response.setContentType("text/plain;charset=UTF-8");
        response.getWriter().println("Session id : " + session.getId());
        response.getWriter().println("New session?: " + session.isNew());
        response.getWriter().println("Visit count: " + count);
        response.getWriter().println("(URL rewriting form: " + response.encodeURL("counter") + ")");
    }
}
```

Expected: `/myapp/counter` baar baar refresh karo → count 1, 2, 3... Dusre browser (ya incognito) me count phir 1 se (alag session).

Session timeout set karne ke 3 tareeke

Kahan	Example
web.xml (app-wide, minutes)	`<session-timeout>30</session-timeout>`
Code (ek session, seconds)	`session.setMaxInactiveInterval(1800)`
Tomcat default	`conf\web.xml` me 30 minute

Cookie band ho to `response.encodeURL("page")` se URL me `;jsessionid=…` jud jaata hai (URL rewriting).

4.4 Register + Login + Logout app

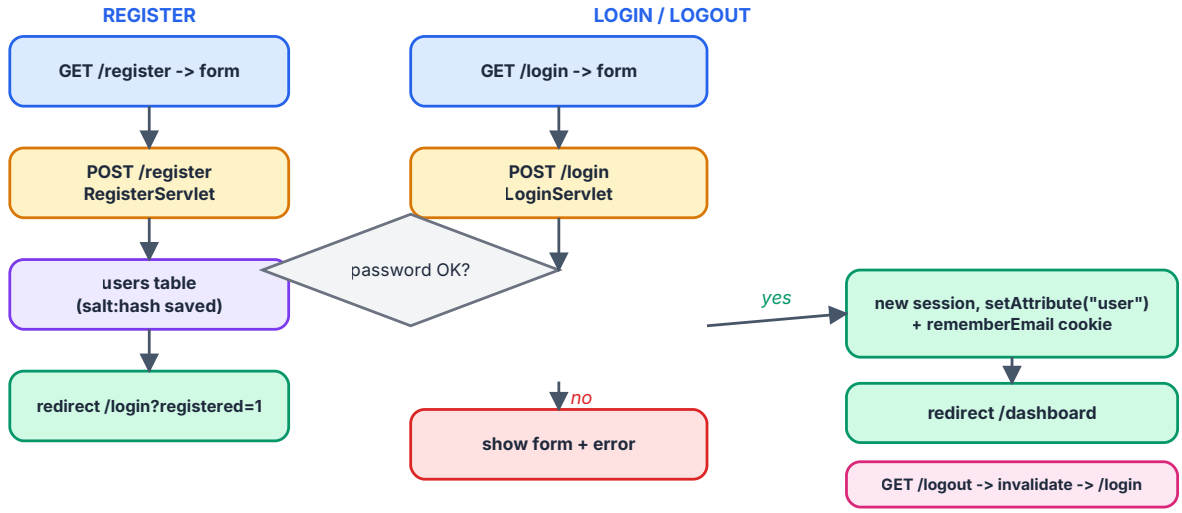


Fig. 30 — Poora flow

Step 1 — Database table (MySQL)

Pehle `C:\myapp-dev\schema.sql` chalao (poora file chapter 5 me hai). Users table:

```sql
CREATE TABLE IF NOT EXISTS users (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(200) NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 2 — Files kahan rakhni hain

File (C:\myapp-dev\src\com\example\...)	Kaam
`model\User.java`	Data class (id, name, email, passwordHash)
`util\PasswordUtil.java`	Password hash + verify
`util\DBUtil.java`	MySQL connection
`util\Html.java`	HTML page template + **esc()** (XSS bachav)
`dao\UserDao.java`	users table ke SQL
`auth\RegisterServlet.java`	/register
`auth\LoginServlet.java`	/login
`auth\DashboardServlet.java`	/dashboard (protected)
`auth\LogoutServlet.java`	/logout
`filter\AuthFilter.java`	Dashboard/students ko protect (ch. 3)

Step 3 — Supporting classes

User.java — User model

```
JAVA · 25 LINES
```

```java
package com.example.model;

public class User {
    private int id;
    private String name;
    private String email;
    private String passwordHash;

    public User() { }

    public User(String name, String email, String passwordHash) {
        this.name = name;
        this.email = email;
        this.passwordHash = passwordHash;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPasswordHash() { return passwordHash; }
    public void setPasswordHash(String passwordHash) { this.passwordHash = passwordHash; }
}
```

PasswordUtil.java — salted SHA-256 — "salt:hash"

JAVA · 35 LINES

```java
package com.example.util;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.util.Base64;

/** Never store plain passwords. Stored format:  salt:hash  (both Base64). */
public final class PasswordUtil {
    private PasswordUtil() { }

    public static String hash(String password) {
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        return Base64.getEncoder().encodeToString(salt) + ":" + digest(salt, password);
    }

    public static boolean verify(String password, String stored) {
        String[] parts = stored.split(":");
        if (parts.length != 2) return false;
        byte[] salt = Base64.getDecoder().decode(parts[0]);
        return digest(salt, password).equals(parts[1]);
    }

    private static String digest(byte[] salt, String password) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            md.update(salt);
            return Base64.getEncoder().encodeToString(md.digest(password.getBytes(StandardCharsets.UTF_8)));
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
}
```

Html.java — page() template + esc()

JAVA · 21 LINES

```java
package com.example.util;

/** Tiny helper so every servlet prints the same page layout. */
```

```
public final class Html {
    private Html() { }

    /** Escape user text before printing it (prevents XSS). */
    public static String esc(String s) {
        if (s == null) return "";
        return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
                .replace("\"", "&quot;").replace("'", "&#39;");
    }

    public static String page(String title, String body) {
        return "<!DOCTYPE html><html><head><meta charset=\"UTF-8\"><title>" + esc(title) + "</title>"
             + "<style>body{font-family:Arial;margin:30px;max-width:720px}"
             + "input,select{padding:6px;margin:4px 0}table{border-collapse:collapse}"
             + "td,th{border:1px solid #999;padding:6px 10px}.err{color:#b00020}.ok{color:#0a7d2c}</style>"
             + "</head><body><h2>" + esc(title) + "</h2>" + body + "</body></html>";
    }
}
```

UserDao.java — insert + findByEmail (PreparedStatement)

JAVA · 43 LINES

```
package com.example.dao;

import com.example.model.User;
import com.example.util.DBUtil;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UserDao {

    /** INSERT a new user. Returns false if the e-mail already exists. */
    public boolean insert(User u) throws SQLException {
        if (findByEmail(u.getEmail()) != null) {
            return false;
        }
        String sql = "INSERT INTO users(name, email, password_hash) VALUES (?, ?, ?)";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, u.getName());
            ps.setString(2, u.getEmail());
            ps.setString(3, u.getPasswordHash());
            return ps.executeUpdate() == 1;
        }
    }

    /** SELECT one user by e-mail (null if not found). */
    public User findByEmail(String email) throws SQLException {
        String sql = "SELECT id, name, email, password_hash FROM users WHERE email = ?";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, email);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    User u = new User(rs.getString("name"), rs.getString("email"),
rs.getString("password_hash"));
                    u.setId(rs.getInt("id"));
                    return u;
                }
                return null;
            }
        }
    }
}
```

Step 4 — Register

RegisterServlet.java /register — GET form, POST save

JAVA · 66 LINES

```java
package com.example.auth;

import com.example.dao.UserDao;
import com.example.model.User;
import com.example.util.Html;
import com.example.util.PasswordUtil;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.sql.SQLException;

@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        showForm(response, "", "", "");
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        String confirm = request.getParameter("confirm");

        if (name == null || name.isBlank() || email == null || email.isBlank()
                || password == null || password.length() < 6) {
            showForm(response, "All fields required; password minimum 6 characters.", name, email);
            return;
        }
        if (!password.equals(confirm)) {
            showForm(response, "Passwords do not match.", name, email);
            return;
        }
        try {
            boolean created = new UserDao().insert(new User(name.trim(), email.trim(),
PasswordUtil.hash(password)));
            if (!created) {
                showForm(response, "This e-mail is already registered.", name, "");
                return;
            }
            // Post-Redirect-Get: browser reload will not re-submit the form
            response.sendRedirect(request.getContextPath() + "/login?registered=1");
        } catch (SQLException e) {
            throw new ServletException("Database error while registering", e);
        }
    }

    private void showForm(HttpServletResponse response, String error, String name, String email) throws
IOException {
        response.setContentType("text/html;charset=UTF-8");
        String body = (error.isEmpty() ? "" : "<p class='err'>" + Html.esc(error) + "</p>")
            + "<form method='post' action='register'>"
            + "Name:<br><input name='name' value='" + Html.esc(name) + "'><br>"
            + "E-mail:<br><input type='email' name='email' value='" + Html.esc(email) + "'><br>"
            + "Password:<br><input type='password' name='password'><br>"
            + "Confirm password:<br><input type='password' name='confirm'><br>"
            + "<input type='submit' value='Register'></form>"
            + "<p>Already registered? <a href='login'>Login</a></p>";
        response.getWriter().println(Html.page("Register", body));
```

```
    }
}
```

- **GET** form dikhata hai, **POST** data save karta hai.
- Password **plain nahi** — `PasswordUtil.hash()` se salt+hash.
- Email pehle se ho to `insert` false deta hai → "already registered" message.
- Success par `/login?registered=1` par redirect.

Step 5 — Login (session + remember-me cookie)

LoginServlet.java — /login — session + cookie dono

JAVA · 82 LINES

```
package com.example.auth;

import com.example.dao.UserDao;
import com.example.model.User;
import com.example.util.Html;
import com.example.util.PasswordUtil;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;
import java.sql.SQLException;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        // READ a cookie: pre-fill the e-mail if "Remember me" was ticked last time
        String remembered = "";
        Cookie[] cookies = request.getCookies();                // can be null!
        if (cookies != null) {
            for (Cookie c : cookies) {
                if ("rememberEmail".equals(c.getName())) {
                    remembered = c.getValue();
                }
            }
        }
        String msg = "";
        if ("1".equals(request.getParameter("registered"))) msg = "<p class='ok'>Registered! Please login.</
p>";
        if ("1".equals(request.getParameter("loggedout")))  msg = "<p class='ok'>You have been logged out.</
p>";
        showForm(response, msg, remembered);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String email = request.getParameter("email");
        String password = request.getParameter("password");
        boolean remember = request.getParameter("remember") != null;   // checkbox: null if unticked

        try {
            User user = (email == null || password == null) ? null : new UserDao().findByEmail(email.trim());
            if (user == null || !PasswordUtil.verify(password, user.getPasswordHash())) {
                showForm(response, "<p class='err'>Wrong e-mail or password.</p>", email == null ? "" :
email);
                return;
            }

            HttpSession old = request.getSession(false);
```

```
            if (old != null) old.invalidate();                    // new session id after login (security)
            HttpSession session = request.getSession(true);       // create fresh session
            session.setAttribute("user", user);                   // login = put user in session
            session.setMaxInactiveInterval(30 * 60);              // 30 minutes of inactivity

            Cookie cookie = new Cookie("rememberEmail", remember ? user.getEmail() : "");
            cookie.setMaxAge(remember ? 7 * 24 * 60 * 60 : 0);    // 7 days, or 0 = delete now
            cookie.setPath(request.getContextPath().isEmpty() ? "/" : request.getContextPath());
            cookie.setHttpOnly(true);                             // JavaScript cannot read it
            response.addCookie(cookie);

            response.sendRedirect(request.getContextPath() + "/dashboard");
        } catch (SQLException e) {
            throw new ServletException("Database error while logging in", e);
        }
    }

    private void showForm(HttpServletResponse response, String msg, String email) throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        String body = msg
            + "<form method='post' action='login'>"
            + "E-mail:<br><input type='email' name='email' value='" + Html.esc(email) + "'><br>"
            + "Password:<br><input type='password' name='password'><br>"
            + "<label><input type='checkbox' name='remember'> Remember my e-mail</label><br>"
            + "<input type='submit' value='Login'></form>"
            + "<p>New user? <a href='register'>Register</a></p>";
        response.getWriter().println(Html.page("Login", body));
    }
}
```

- **Login = session.setAttribute("user", user)**. Itna hi.
- Login se pehle purana session **invalidate** — session fixation attack se bachav.
- “Remember me” → `rememberEmail` cookie 7 din; untick karo to `setMaxAge(0)` se delete.

Step 6 — Dashboard (protected page)

DashboardServlet.java — /dashboard — session info + visit counter

JAVA · 46 LINES

```
package com.example.auth;

import com.example.listener.AppListener;
import com.example.model.User;
import com.example.util.Html;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Date;

/** Private page. AuthFilter guarantees a user is in the session before we get here. */
@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession();
        User user = (User) session.getAttribute("user");

        Integer visits = (Integer) session.getAttribute("visits");   // per-user counter
        visits = (visits == null) ? 1 : visits + 1;
        session.setAttribute("visits", visits);

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
```

```
        out.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><title>Dashboard</title></head><body>");

        request.getRequestDispatcher("/header").include(request, response);   // INCLUDE shared menu

        out.println("<h2>Welcome, " + Html.esc(user.getName()) + "</h2>");
        out.println("<table>");
        out.println("<tr><td>Session id</td><td>" + session.getId() + "</td></tr>");
        out.println("<tr><td>Created</td><td>" + new Date(session.getCreationTime()) + "</td></tr>");
        out.println("<tr><td>Last accessed</td><td>" + new Date(session.getLastAccessedTime()) + "</td></
tr>");
        out.println("<tr><td>Timeout (seconds)</td><td>" + session.getMaxInactiveInterval() + "</td></tr>");
        out.println("<tr><td>Your visits this session</td><td>" + visits + "</td></tr>");
        out.println("<tr><td>Active sessions on server</td><td>" + AppListener.ACTIVE_SESSIONS.get() + "</
td></tr>");
        out.println("</table></body></html>");
    }
}
```

Step 7 — Logout

LogoutServlet.java — /logout — session.invalidate()

JAVA · 23 LINES

```java
package com.example.auth;

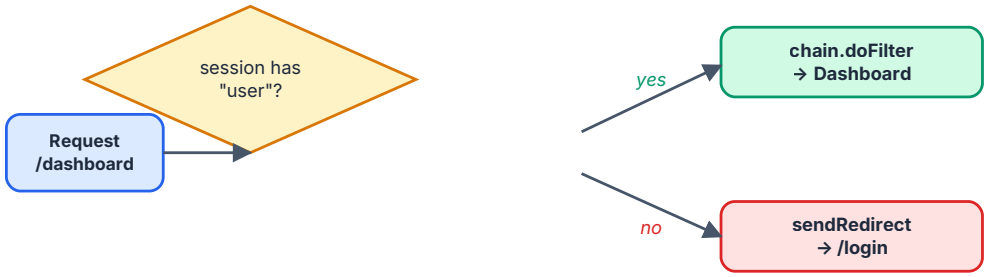
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate();            // removes the user and every attribute; session is dead
        }
        response.sendRedirect(request.getContextPath() + "/login?loggedout=1");
    }
}
```

Test sequence

1. `/myapp/dashboard` kholo (bina login) → **/login par redirect** (AuthFilter).
2. `/myapp/register` → naam, email, password → Register → login page par “Registered!” message.
3. Login (Remember tick) → **/dashboard** par welcome + session id + visits.
4. Logout → login page “You have been logged out”. Back button dabao → protected page nahi dikhta (no-store header).
5. Login page dubara kholo → email pre-filled (cookie).



One filter protects many pages — no check code inside each servlet.

Fig. 31 — Dashboard ko AuthFilter kaise bachata hai

Risk	Fix used in our code
Plain-text passwords	salted SHA-256 (BCrypt in production)
SQL injection	PreparedStatement with ?
XSS (script in input)	Html.esc() on every output
Session fixation	invalidate old session, create new at login
Cookie theft by script	setHttpOnly(true)
Back button after logout	Cache-Control: no-store

Fig. 32 — Security checklist — humare code me kya use hua

⚠️ Careful — Production note

SHA-256 + salt seekhne ke liye theek hai. Asli project me **BCrypt / Argon2** use karo aur HTTPS lagao (cookie me `Secure` flag).

❓ Likely exam questions — Chapter 4 — Likely exam questions

1. Session tracking ke techniques (cookie, hidden field, URL rewriting, HttpSession) compare karo.
2. Cookie kya hai? Create, read, delete ka code likho. setMaxAge(0) ka matlab?
3. HttpSession kaise kaam karti hai? JSESSIONID kya hai?
4. getSession() aur getSession(false) me difference.
5. Cookie aur Session me difference (table).
6. Login / logout servlet likho (session ke saath).
7. Session timeout kaise set karte hain (web.xml, code)?
8. Session fixation, XSS, SQL injection se kaise bachte hain?

CHAPTER 5

JDBC & MySQL CRUD

JDBC architecture | MySQL setup | 7 steps | Statement types | CRUD | Transaction | Servlet + JDBC | Connection pool

5.1 JDBC kya hai?

📘 **Definition**

JDBC (Java Database Connectivity) = Java ka standard API (`java.sql.*`) jisse Java program kisi bhi relational database (MySQL, Oracle...) se connect hoke SQL chala sakta hai.

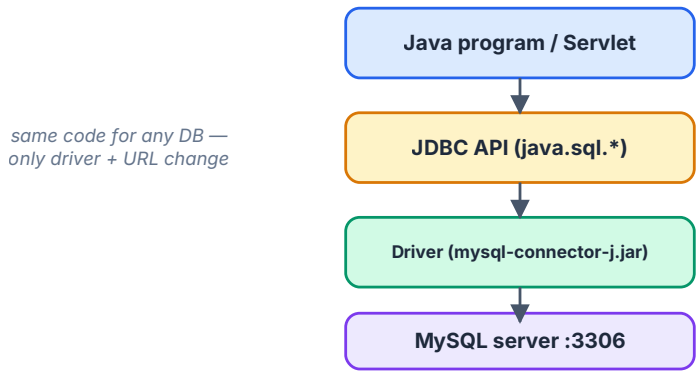
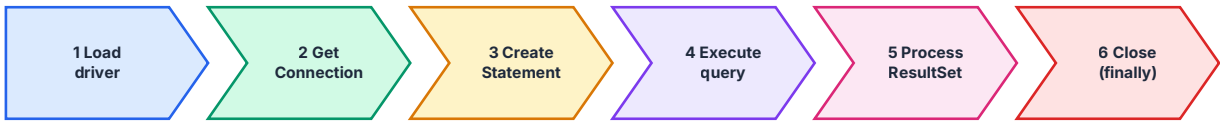


Fig. 33 — JDBC architecture

Type	Name	How	Today
1	JDBC-ODBC bridge	via ODBC	removed (Java 8)
2	Native-API	DB client library	rare
3	Network protocol	middleware server	rare
4	Thin (pure Java)	direct socket to DB	USED — MySQL Connector/J

Fig. 34 — JDBC driver ke 4 types



Use try-with-resources: Connection, Statement, ResultSet close automatically.

Class.forName(...) is optional with JDBC 4+ (jar in classpath is enough).

Fig. 35 — JDBC ke steps

Important interface	Kaam
`DriverManager`	Driver dhundhke Connection deta hai
`Connection`	Database se session
`Statement / PreparedStatement`	SQL chalane ke liye
`ResultSet`	SELECT ke rows
`SQLException`	Sab DB errors

5.2 MySQL + driver setup (Windows)

1. MySQL Server 8 install karo, root password yaad rakho (humne `root123` use kiya — apna password `DBUtil.java` me badlo).
2. Connector/J (`mysql-connector-j-8.x.jar`) download karo (dev.mysql.com → Connector/J → Platform Independent).
3. Jar ko `C:\jdbc-practice` me rakho.
4. Tables banao:

```bash
mysql -u root -p < C:\myapp-dev\schema.sql
```

schema.sql — C:\myapp-dev\schema.sql — database + tables + sample rows

```sql
-- Run in MySQL:   mysql -u root -p < schema.sql
CREATE DATABASE IF NOT EXISTS college CHARACTER SET utf8mb4;
USE college;

CREATE TABLE IF NOT EXISTS users (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(200) NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS students (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    name    VARCHAR(100) NOT NULL,
    email   VARCHAR(150) NOT NULL,
    course  VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS accounts (          -- used by the transaction demo
    acc_no  INT PRIMARY KEY,
```

```
    holder  VARCHAR(100),
    balance DECIMAL(10,2)
);

INSERT INTO students(name, email, course) VALUES
    ('Asha',  'asha@example.com',  'MCA'),
    ('Ravi',  'ravi@example.com',  'M.Tech'),
    ('Meena', 'meena@example.com', 'MCA');

INSERT INTO accounts VALUES (101, 'Asha', 5000.00), (102, 'Ravi', 3000.00);

DELIMITER $$
CREATE PROCEDURE get_student(IN p_id INT, OUT p_name VARCHAR(100))
BEGIN
    SELECT name INTO p_name FROM students WHERE id = p_id;
END$$
DELIMITER ;
```

jdbc:mysql://localhost:3306/college?serverTimezone=UTC

jdbc:mysql	localhost	3306	college	?serverTimezone=UTC
protocol	host	port	database	options

Fig. 36 — Connection URL

> ⚠️ **Careful — Common JDBC errors**
>
> **No suitable driver / ClassNotFoundException** → jar classpath me nahi. **Access denied for user** → username/password galat. **Communications link failure** → MySQL band hai ya port galat. **Unknown database** → schema.sql nahi chalaya. **Public Key Retrieval is not allowed** → URL me `&allowPublicKeyRetrieval=true` jodo.

5.3 Pehla JDBC program (7 steps)

ConnectTest.java C:\jdbc-practice\ConnectTest.java

JAVA · 43 LINES

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

/** The 7 JDBC steps in one small program. */
public class ConnectTest {
    public static void main(String[] args) {
        String url  = "jdbc:mysql://localhost:3306/college?serverTimezone=UTC";
        String user = "root";
        String pass = "root123";

        try {
            // Step 1: load the driver (optional with JDBC 4+, but good for exams)
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: open a connection
            Connection con = DriverManager.getConnection(url, user, pass);
            System.out.println("Connected to " + con.getMetaData().getDatabaseProductName());

            // Step 3: create a statement
            Statement st = con.createStatement();

            // Step 4: execute the query
            ResultSet rs = st.executeQuery("SELECT id, name, course FROM students");
```

```
            // Step 5: process the ResultSet (cursor starts BEFORE the first row)
            while (rs.next()) {
                System.out.println(rs.getInt("id") + "  " + rs.getString("name") + "  " + rs.getString(3));
            }

            // Step 6: close everything (reverse order)
            rs.close();
            st.close();
            con.close();
        } catch (ClassNotFoundException e) {
            System.out.println("Driver jar not on classpath: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL error: " + e.getMessage() + "  (code " + e.getErrorCode() + ")");
        }
    }
}
```

BASH

```
cd C:\jdbc-practice
javac ConnectTest.java
java -cp ".;mysql-connector-j-8.3.0.jar" ConnectTest
```

(Jar ka naam apne download ke hisaab se likho; Windows me classpath separator **;** hota hai, Linux/Mac me **:**.)
Expected: “Connected!” aur students ke naam print.

	Statement	PreparedStatement	CallableStatement
Use	fixed SQL	SQL with ? parameters	stored procedures
SQL injection	VULNERABLE	SAFE	safe
Speed	compiled each time	precompiled, reusable	on server
Call	executeQuery(sql)	setString(1,x)	{call proc(?,?)}

Fig. 37 — Statement vs PreparedStatement vs CallableStatement

Method	For	Returns
executeQuery()	SELECT	ResultSet
executeUpdate()	INSERT / UPDATE / DELETE	int (rows affected)
execute()	any / unknown	boolean

Fig. 38 — execute methods

ResultSet = cursor over rows

id	name	email	course
1	Rohit	r@x.com	M.Tech
2	Amit	a@x.com	B.Tech

```
while (rs.next()) {
 rs.getInt("id");
 rs.getString("name");
}
```

Cursor starts BEFORE the first row — next() moves to row 1.

Fig. 39 — ResultSet

🧠 **Yaad rakho**

Hamesha **PreparedStatement** with `?` use karo — **SQL injection** se bachata hai. User input ko SQL string me `+` se kabhi mat jodo.

5.4 CRUD — console program

CRUD	SQL	JDBC call	HTTP
Create	INSERT	executeUpdate	POST /students/save
Read	SELECT	executeQuery	GET /students
Update	UPDATE	executeUpdate	POST /students/save (id)
Delete	DELETE	executeUpdate	POST /students/delete

Fig. 40 — CRUD ka mapping

StudentCrud.java C:\jdbc-practice\StudentCrud.java — menu: Add / List / Update / Delete

JAVA · 102 LINES

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Scanner;

/** Console menu: Create / Read / Update / Delete on the students table. */
public class StudentCrud {
    static final String URL  = "jdbc:mysql://localhost:3306/college?serverTimezone=UTC";
    static final String USER = "root";
    static final String PASS = "root123";

    static Connection connect() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }

    // C - INSERT
    static void create(String name, String email, String course) throws SQLException {
        String sql = "INSERT INTO students(name, email, course) VALUES (?, ?, ?)";
        try (Connection con = connect(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, name);
            ps.setString(2, email);
            ps.setString(3, course);
            int rows = ps.executeUpdate();                // returns number of rows changed
```

```
            System.out.println(rows + " row inserted.");
        }
    }

    // R - SELECT
    static void readAll() throws SQLException {
        String sql = "SELECT id, name, email, course FROM students ORDER BY id";
        try (Connection con = connect();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            System.out.printf("%-4s %-15s %-25s %-10s%n", "ID", "NAME", "EMAIL", "COURSE");
            while (rs.next()) {
                System.out.printf("%-4d %-15s %-25s %-10s%n",
                        rs.getInt("id"), rs.getString("name"), rs.getString("email"), rs.getString("course"));
            }
        }
    }

    // U - UPDATE
    static void update(int id, String course) throws SQLException {
        String sql = "UPDATE students SET course = ? WHERE id = ?";
        try (Connection con = connect(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, course);
            ps.setInt(2, id);
            int rows = ps.executeUpdate();
            System.out.println(rows == 0 ? "No student with id " + id : rows + " row updated.");
        }
    }

    // D - DELETE
    static void delete(int id) throws SQLException {
        String sql = "DELETE FROM students WHERE id = ?";
        try (Connection con = connect(); PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            int rows = ps.executeUpdate();
            System.out.println(rows == 0 ? "No student with id " + id : rows + " row deleted.");
        }
    }

    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n1.Add  2.List  3.Update course  4.Delete  5.Exit");
                System.out.print("Choice: ");
                int choice = Integer.parseInt(sc.nextLine().trim());
                try {
                    switch (choice) {
                        case 1:
                            System.out.print("Name: ");   String n = sc.nextLine();
                            System.out.print("Email: ");  String e = sc.nextLine();
                            System.out.print("Course: "); String c = sc.nextLine();
                            create(n, e, c);
                            break;
                        case 2:
                            readAll();
                            break;
                        case 3:
                            System.out.print("Id: ");         int uid =
Integer.parseInt(sc.nextLine().trim());
                            System.out.print("New course: "); update(uid, sc.nextLine());
                            break;
                        case 4:
                            System.out.print("Id: ");
                            delete(Integer.parseInt(sc.nextLine().trim()));
                            break;
                        case 5:
                            System.out.println("Bye!");
                            return;
                        default:
                            System.out.println("Choose 1-5.");
                    }
                } catch (SQLException ex) {
                    System.out.println("Database error: " + ex.getMessage());
```

```
            }
        }
    }
}
```

```bash
javac StudentCrud.java
java -cp ".;mysql-connector-j-8.3.0.jar" StudentCrud
```

5.5 Transaction, Batch, Stored procedure

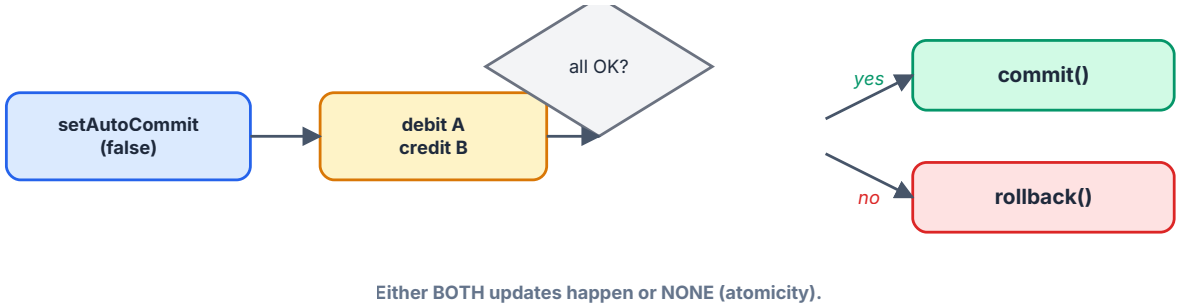


Fig. 41 — Transaction — commit / rollback

TransferDemo.java — account transfer — setAutoCommit(false)

JAVA · 33 LINES

```java
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

/** Transaction: move money from one account to another - BOTH updates succeed or NONE. */
public class TransferDemo {
    public static void main(String[] args) throws SQLException {
        String url = "jdbc:mysql://localhost:3306/college?serverTimezone=UTC";
        try (Connection con = DriverManager.getConnection(url, "root", "root123")) {
            con.setAutoCommit(false);                    // start a transaction
            try (PreparedStatement debit  = con.prepareStatement("UPDATE accounts SET balance = balance - ?
WHERE acc_no = ?");
                 PreparedStatement credit = con.prepareStatement("UPDATE accounts SET balance = balance + ?
WHERE acc_no = ?")) {

                BigDecimal amount = new BigDecimal("500.00");
                debit.setBigDecimal(1, amount);
                debit.setInt(2, 101);
                debit.executeUpdate();

                credit.setBigDecimal(1, amount);
                credit.setInt(2, 102);
                credit.executeUpdate();

                con.commit();                            // make both changes permanent
                System.out.println("Transfer done.");
            } catch (SQLException e) {
                con.rollback();                          // undo everything since setAutoCommit(false)
                System.out.println("Transfer failed, rolled back: " + e.getMessage());
            }
        }
    }
}
```

BatchDemo.java — addBatch / executeBatch

JAVA · 25 LINES

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

/** Batch: send many INSERTs together (much faster than one by one). */
public class BatchDemo {
    public static void main(String[] args) throws SQLException {
        String url = "jdbc:mysql://localhost:3306/college?serverTimezone=UTC";
        String sql = "INSERT INTO students(name, email, course) VALUES (?, ?, ?)";
        String[][] data = { {"Kiran", "kiran@example.com", "MCA"}, {"Neha", "neha@example.com", "BCA"},
{"Omkar", "omkar@example.com", "M.Tech"} };

        try (Connection con = DriverManager.getConnection(url, "root", "root123");
             PreparedStatement ps = con.prepareStatement(sql)) {
            for (String[] row : data) {
                ps.setString(1, row[0]);
                ps.setString(2, row[1]);
                ps.setString(3, row[2]);
                ps.addBatch();                      // queue it
            }
            int[] counts = ps.executeBatch();       // send all at once
            System.out.println(counts.length + " statements executed.");
        }
    }
}
```

ProcedureDemo.java — CallableStatement — {call get_student(?,?)

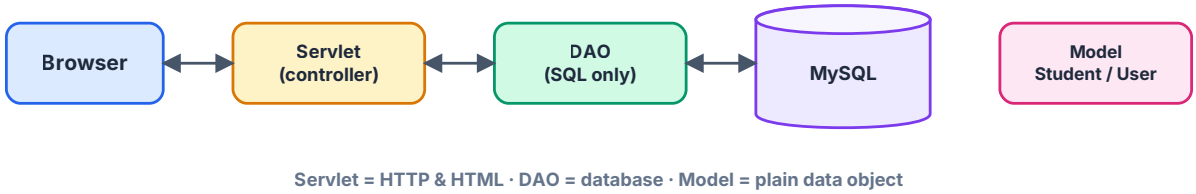
JAVA · 19 LINES

```java
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Types;

/** CallableStatement: call the stored procedure get_student(IN id, OUT name). */
public class ProcedureDemo {
    public static void main(String[] args) throws SQLException {
        String url = "jdbc:mysql://localhost:3306/college?serverTimezone=UTC";
        try (Connection con = DriverManager.getConnection(url, "root", "root123");
             CallableStatement cs = con.prepareCall("{call get_student(?, ?)}")) {
            cs.setInt(1, 1);                        // IN parameter
            cs.registerOutParameter(2, Types.VARCHAR);   // OUT parameter
            cs.execute();
            System.out.println("Student 1 is " + cs.getString(2));
        }
    }
}
```

}

5.6 Servlet me database (MVC style)



Servlet = HTTP & HTML · DAO = database · Model = plain data object

Fig. 42 — Servlet → DAO → MySQL

DBUtil.java — ek jagah connection logic

JAVA · 27 LINES

```java
package com.example.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/** One place that knows how to connect to MySQL. */
public final class DBUtil {
    private static final String URL =
        "jdbc:mysql://localhost:3306/college?serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "root123";      // change to YOUR MySQL password

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");      // load the driver class
        } catch (ClassNotFoundException e) {
            throw new ExceptionInInitializerError("MySQL driver jar missing in WEB-INF/lib: " + e);
        }
    }

    private DBUtil() { }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Student.java — model

JAVA · 26 LINES

```java
package com.example.model;

public class Student {
    private int id;
    private String name;
    private String email;
    private String course;

    public Student() { }

    public Student(int id, String name, String email, String course) {
        this.id = id;
        this.name = name;
        this.email = email;
```

```
        this.course = course;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getCourse() { return course; }
    public void setCourse(String course) { this.course = course; }
}
```

StudentDao.java — CRUD ke SQL

JAVA · 82 LINES

```
package com.example.dao;

import com.example.model.Student;
import com.example.util.DBUtil;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;

/** CRUD for the students table. */
public class StudentDao {

    // C - Create
    public int insert(Student s) throws SQLException {
        String sql = "INSERT INTO students(name, email, course) VALUES (?, ?, ?)";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, s.getName());
            ps.setString(2, s.getEmail());
            ps.setString(3, s.getCourse());
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                return keys.next() ? keys.getInt(1) : 0;      // new auto-increment id
            }
        }
    }

    // R - Read (all)
    public List<Student> findAll() throws SQLException {
        List<Student> list = new ArrayList<>();
        String sql = "SELECT id, name, email, course FROM students ORDER BY id";
        try (Connection con = DBUtil.getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                list.add(new Student(rs.getInt("id"), rs.getString("name"),
                                     rs.getString("email"), rs.getString("course")));
            }
        }
        return list;
    }

    // R - Read (one)
    public Student findById(int id) throws SQLException {
        String sql = "SELECT id, name, email, course FROM students WHERE id = ?";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next()
                    ? new Student(rs.getInt("id"), rs.getString("name"), rs.getString("email"),
rs.getString("course"))
                    : null;
```

```
            }
        }
    }

    // U - Update
    public boolean update(Student s) throws SQLException {
        String sql = "UPDATE students SET name = ?, email = ?, course = ? WHERE id = ?";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, s.getName());
            ps.setString(2, s.getEmail());
            ps.setString(3, s.getCourse());
            ps.setInt(4, s.getId());
            return ps.executeUpdate() > 0;
        }
    }

    // D - Delete
    public boolean delete(int id) throws SQLException {
        String sql = "DELETE FROM students WHERE id = ?";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            return ps.executeUpdate() > 0;
        }
    }
}
```

StudentServlet.java — /students — list, new, edit, insert, update, delete

JAVA · 104 LINES

```
package com.example.crud;

import com.example.dao.StudentDao;
import com.example.model.Student;
import com.example.util.Html;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.SQLException;
import java.util.List;

/**
 *  GET  /students            -> list all     (Read)
 *  GET  /students?action=new -> empty form
 *  GET  /students?action=edit&id=3 -> form filled with student 3
 *  POST /students            -> action=insert | update | delete
 */
@WebServlet("/students")
public class StudentServlet extends HttpServlet {
    private final StudentDao dao = new StudentDao();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String action = request.getParameter("action");
        try {
            if ("new".equals(action)) {
                showForm(request, response, new Student(), "insert");
            } else if ("edit".equals(action)) {
                Student s = dao.findById(Integer.parseInt(request.getParameter("id")));
                if (s == null) { response.sendError(HttpServletResponse.SC_NOT_FOUND, "No such student");
return; }
                showForm(request, response, s, "update");
            } else {
                showList(request, response);
            }
        } catch (SQLException | NumberFormatException e) {
```

```
            throw new ServletException(e);
        }
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String action = request.getParameter("action");
        try {
            if ("delete".equals(action)) {
                dao.delete(Integer.parseInt(request.getParameter("id")));
            } else {
                Student s = new Student();
                s.setName(request.getParameter("name"));
                s.setEmail(request.getParameter("email"));
                s.setCourse(request.getParameter("course"));
                if ("update".equals(action)) {
                    s.setId(Integer.parseInt(request.getParameter("id")));
                    dao.update(s);
                } else {
                    dao.insert(s);
                }
            }
            response.sendRedirect(request.getContextPath() + "/students");   // Post-Redirect-Get
        } catch (SQLException | NumberFormatException e) {
            throw new ServletException(e);
        }
    }

    private void showList(HttpServletRequest request, HttpServletResponse response)
            throws SQLException, ServletException, IOException {
        List<Student> list = dao.findAll();
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<!DOCTYPE html><html><head><meta charset='UTF-8'><title>Students</title>"
                  + "<style>body{font-family:Arial;margin:30px}table{border-collapse:collapse}"
                  + "td,th{border:1px solid #999;padding:6px 10px}</style></head><body>");
        request.getRequestDispatcher("/header").include(request, response);
        out.println("<h2>Students</h2><p><a href='students?action=new'>+ Add student</a></p>");
        out.println("<table><tr><th>ID</th><th>Name</th><th>E-mail</th><th>Course</th><th>Actions</th></tr>");
        for (Student s : list) {
            out.println("<tr><td>" + s.getId() + "</td><td>" + Html.esc(s.getName()) + "</td><td>"
                + Html.esc(s.getEmail()) + "</td><td>" + Html.esc(s.getCourse()) + "</td><td>"
                + "<a href='students?action=edit&id=" + s.getId() + "'>Edit</a> "
                + "<form style='display:inline' method='post' action='students' onsubmit=\"return
confirm('Delete?')\">"
                + "<input type='hidden' name='action' value='delete'><input type='hidden' name='id' value='"
+ s.getId() + "'>"
                + "<input type='submit' value='Delete'></form></td></tr>");
        }
        out.println("</table></body></html>");
    }

    private void showForm(HttpServletRequest request, HttpServletResponse response, Student s, String action)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        String body = "<form method='post' action='students'>"
            + "<input type='hidden' name='action' value='" + action + "'>"
            + "<input type='hidden' name='id' value='" + s.getId() + "'>"
            + "Name:<br><input name='name' value='" + Html.esc(s.getName()) + "'><br>"
            + "E-mail:<br><input name='email' value='" + Html.esc(s.getEmail()) + "'><br>"
            + "Course:<br><input name='course' value='" + Html.esc(s.getCourse()) + "'><br>"
            + "<input type='submit' value='Save'> <a href='students'>Cancel</a></form>";
        response.getWriter().println(Html.page("update".equals(action) ? "Edit student" : "Add student",
body));
    }
}
```

URL	Kya hota hai
`GET /myapp/students`	Table me saare students (Read)
`GET /students?action=new`	Add form
`POST action=insert`	Naya row (Create) → redirect
`GET /students?action=edit&id=2`	Purani values bhara form
`POST action=update`	Row update
`POST action=delete`	Row delete (confirm ke baad)

Jar kahan? DriverManager (DBUtil) ke liye `C:\tomcat\webapps\myapp\WEB-INF\lib\mysql-connector-j-8.x.jar`. Jar copy karne ke baad Tomcat restart. Students page ke liye pehle **login** chahiye (AuthFilter).

Where is the connector jar?	Use when
WEB-INF\lib\ (inside your app)	DriverManager.getConnection (DBUtil)
%CATALINA_HOME%\lib\	JNDI DataSource pool from context.xml
same folder / -cp for console	java -cp ".;mysql-connector-j.jar" Class

Never put it in BOTH lib folders — it causes class-loading errors.

Fig. 43 — Jar kahan rakhein

5.7 Connection pool (context.xml + JNDI)

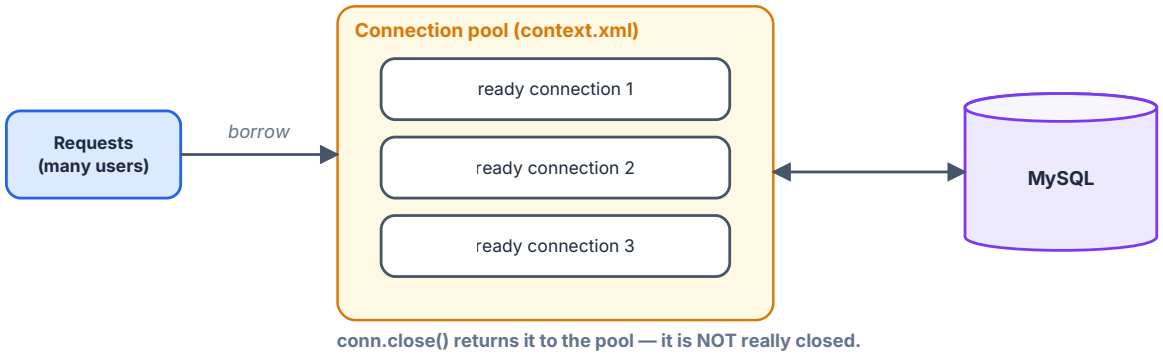


Fig. 44 — Connection pool

JndiDB.java — DataSource ko JNDI se lookup

JAVA · 27 LINES

```java
package com.example.util;

import java.sql.Connection;
import java.sql.SQLException;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.naming.NamingException;
```

```java
import javax.sql.DataSource;

/** Alternative to DBUtil: use the connection POOL that Tomcat creates from META-INF/context.xml. */
public final class JndiDB {
    private static DataSource ds;

    private JndiDB() { }

    public static Connection getConnection() throws SQLException {
        try {
            if (ds == null) {
                Context ctx = new InitialContext();
                ds = (DataSource) ctx.lookup("java:comp/env/jdbc/college");   // name from context.xml
            }
            return ds.getConnection();        // borrowed from the pool; close() returns it
        } catch (NamingException e) {
            throw new SQLException("DataSource lookup failed", e);
        }
    }
}
```

1. context.xml me `<Resource name="jdbc/college" …>` (chapter 2.4).
2. web.xml me `<resource-ref>` (chapter 2.2).
3. Connector jar ko `C:\tomcat\lib` me rakho (webapp ke WEB-INF\lib me **nahi**).
4. DAO me `DBUtil.getConnection()` ki jagah `JndiDB.getConnection()`.

⭐ Exam tip — Pool kyon?

Har request par naya connection banana **slow** hai. Pool pehle se connections taiyar rakhta hai; `close()` se connection pool me wapas jaata hai.

❓ Likely exam questions — Chapter 5 — Likely exam questions

1. JDBC kya hai? Architecture aur 4 driver types samjhao.
2. JDBC ke steps likho aur MySQL se connect karne ka program do.
3. Statement, PreparedStatement, CallableStatement me difference.
4. CRUD (insert, select, update, delete) ke JDBC programs likho.
5. ResultSet kya hai? executeQuery / executeUpdate / execute me difference.
6. Transaction management (commit, rollback) example ke saath.
7. Servlet se database connect karke students ki list dikhao.
8. Connection pooling aur DataSource kya hai?
9. SQL injection kya hai? PreparedStatement kaise rokta hai?

CHAPTER 6

Applet Programs

Lifecycle | Hello applet | Parameters | Events | Graphics | Run kaise karein | Sandbox

6.1 Applet kya hai?

📘 **Definition**

Applet ek chhota Java GUI program hota tha jo web page (`<applet>` tag) ke andar browser me chalta tha. Class `java.applet.Applet` ya `javax.swing.JApplet` extend karti hai. **main() nahi hota**.

⚠️ **Careful — Applet ab deprecated hai**

Browsers ne applet support band kar diya, **appletviewer JDK 11 me hata diya gaya** aur `java.applet` "removal ke liye marked" hai. Phir bhi exams me aata hai, isliye yahan **AppletRunner** (Frame me applet chalane wala launcher) diya hai — modern JDK par chalta hai. Ya JDK 8 install karke `appletviewer` use karo.

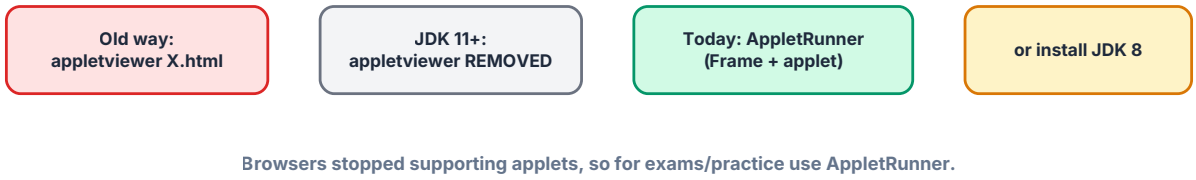


Fig. 48 — Applet chalane ke tareeke



Fig. 49 — Applet lifecycle

Method	Kab
`init()`	Ek baar, load hone par — variables/components setup
`start()`	Page visible hone par (baar baar ho sakta hai)
`paint(Graphics g)`	Screen par draw (**paint**, "pain" nahi!)
`stop()`	Page chhodne / minimize par
`destroy()`	Applet hatne par — resources free

6.2 Programs

1. Hello Applet

HelloApplet.java C:\applets\HelloApplet.java

JAVA · 15 LINES

```java
import java.applet.Applet;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;

/** The smallest applet. paint() draws on the applet's screen area. */
public class HelloApplet extends Applet {

    @Override
    public void paint(Graphics g) {              // NOTE: the method is paint, not pain!
        g.setColor(Color.BLUE);
        g.setFont(new Font("Arial", Font.BOLD, 20));
        g.drawString("Hello, Applet!", 50, 60);   // text, x, y (y = baseline)
    }
}
```



Fig. 45 — HelloApplet ka output (asli run ka screenshot)

2. Parameters (<param>) padhna

ParamApplet.java ParamApplet — getParameter()

JAVA · 23 LINES

```java
import java.applet.Applet;
import java.awt.Graphics;

/** Reads values given in the HTML file with <param name=... value=...>. */
public class ParamApplet extends Applet {
    private String user;
```

```
    private int times;

    @Override
    public void init() {
        user = getParameter("user");                    // returns null if not given
        if (user == null) user = "Guest";
        String t = getParameter("times");
        times = (t == null) ? 1 : Integer.parseInt(t);
    }

    @Override
    public void paint(Graphics g) {
        for (int i = 0; i < times; i++) {
            g.drawString("Welcome, " + user + "  (" + (i + 1) + ")", 20, 30 + i * 20);
        }
    }
}
```

hello.html — hello.html — applet tag + param

HTML · 10 LINES

```
<html>
<body>
  <h2>Applet demo</h2>
  <applet code="ParamApplet.class" width="300" height="120">
    <param name="user"  value="Rohit">
    <param name="times" value="3">
    Your browser does not support applets.
  </applet>
</body>
</html>
```

3. Button + lifecycle events

ClickApplet.java — ClickApplet — ActionListener

JAVA · 45 LINES

```
import java.applet.Applet;
import java.awt.Button;
import java.awt.Graphics;
import java.awt.Label;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

/** Applet life cycle + button event handling + repaint(). */
public class ClickApplet extends Applet implements ActionListener {
    private Button button;
    private Label label;
    private int clicks = 0;
    private final StringBuilder log = new StringBuilder();

    @Override
    public void init() {                        // 1. once, when the applet is loaded
        log.append("init ");
        button = new Button("Click me");
        label = new Label("Clicks: 0     ");
        button.addActionListener(this);         // register the listener
        add(button);
        add(label);
    }

    @Override
    public void start() { log.append("start "); }     // 2. every time the page becomes visible

    @Override
    public void stop() { log.append("stop "); }       // 3. when the page is hidden

    @Override
    public void destroy() { log.append("destroy "); } // 4. once, when the applet is removed
```

```
    @Override
    public void actionPerformed(ActionEvent e) {
        clicks++;
        label.setText("Clicks: " + clicks);
        repaint();                                  // asks AWT to call paint() again
    }

    @Override
    public void paint(Graphics g) {
        g.drawString("Life cycle so far: " + log, 10, 80);
    }
}
```

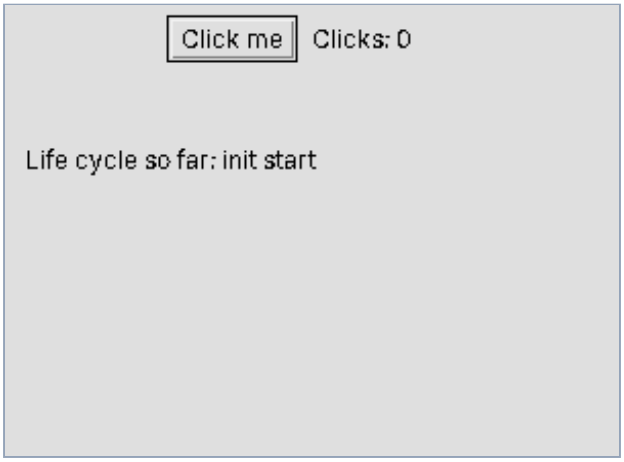


Fig. 46 — ClickApplet — click count aur lifecycle

4. Graphics draw karna

DrawApplet.java — DrawApplet — line, rect, oval, text

JAVA · 22 LINES

```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Graphics;

/** Basic graphics: line, rectangle, oval, filled shapes, colours. */
public class DrawApplet extends Applet {

    @Override
    public void paint(Graphics g) {
        g.setColor(Color.BLACK);
        g.drawLine(10, 10, 200, 10);
        g.drawRect(10, 20, 100, 50);              // x, y, width, height
        g.setColor(Color.RED);
        g.fillRect(130, 20, 70, 50);
        g.setColor(Color.GREEN);
        g.drawOval(10, 90, 100, 60);
        g.setColor(Color.BLUE);
        g.fillOval(130, 90, 70, 60);
        g.setColor(Color.MAGENTA);
        g.drawString("Shapes drawn with Graphics", 10, 175);
    }
}
```

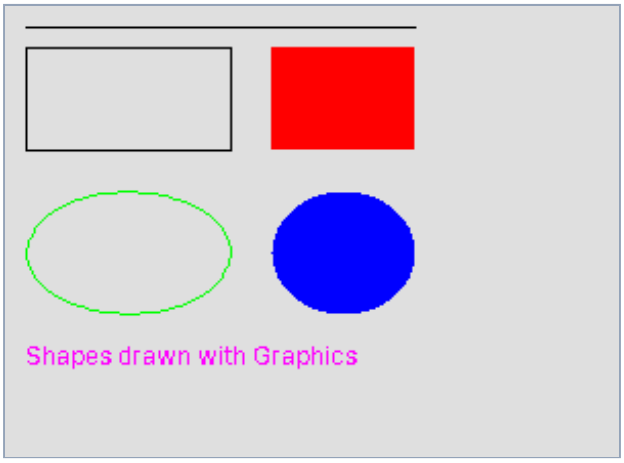


Fig. 47 — DrawApplet ka output

6.3 Kaise chalayein

Option A — AppletRunner (JDK 11/17/21)

AppletRunner.java — AppletRunner — applet ko Frame me chalata hai

JAVA · 30 LINES

```java
import java.applet.Applet;
import java.awt.Frame;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

/**
 * Modern JDKs have no appletviewer and browsers cannot run applets.
 * This launcher puts an applet inside a normal window:   java AppletRunner ClickApplet
 */
public class AppletRunner {
    public static void main(String[] args) throws Exception {
        String className = args.length > 0 ? args[0] : "HelloApplet";
        Applet applet = (Applet) Class.forName(className).getDeclaredConstructor().newInstance();

        Frame frame = new Frame(className);
        frame.add(applet);
        frame.setSize(300, 220);
        frame.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                applet.stop();
                applet.destroy();
                frame.dispose();
            }
        });
        applet.init();            // same order the browser would use
        applet.start();
        frame.setVisible(true);
    }
}
```

BASH

```bash
cd C:\applets
javac *.java
java AppletRunner HelloApplet
java AppletRunner ClickApplet
java AppletRunner DrawApplet
```

Javac “deprecated / removal” warning dega — normal hai, program phir bhi chalega.

Option B — JDK 8 appletviewer

```bash
javac HelloApplet.java
appletviewer hello.html
```

HTML me `code="ParamApplet.class"` jaisi class ka naam; `width/height` pixel me.

6.4 Applet restrictions & comparison

Applet CANNOT	Applet CAN
read/write local files	draw graphics, handle events
connect to other hosts (only origin)	get <param> values
run local programs	play sound / images from its server

Fig. 50 — Sandbox restrictions

	Applet	Application	Servlet
Entry	init()	main()	doGet()/doPost()
Runs in	browser (JVM plugin)	own JVM	Tomcat (server)
UI	AWT/Swing	any	HTML
Status	DEAD — removed	alive	alive

Fig. 51 — Applet vs Application vs Servlet

Applet	Servlet
Client (browser) par chalta	Server (Tomcat) par chalta
GUI (AWT/Swing)	HTML response
init/start/paint/stop/destroy	init/service/destroy
Security restricted	Server ke resources par poori access

❓ Likely exam questions — Chapter 6 — Likely exam questions

1. Applet kya hai? Lifecycle methods diagram ke saath samjhao.
2. Hello World applet likho aur HTML se kaise chalate hain batao.
3. Applet me parameters kaise pass karte hain (<param>, getParameter)?
4. Applet aur application me difference. Applet ki security restrictions.
5. Button click handle karne wala applet likho.
6. Applet aur servlet me difference.

Troubleshooting — error aaye to

Error	Wajah → Fix
404 Not Found	URL galat / app deploy nahi hui. Context path (`/myapp`) aur `@WebServlet` path check karo. `webapps\myapp\WEB-INF\classes` me .class hai?
405 Method Not Allowed	POST bheja par servlet me sirf `doGet` hai (ya ulta). Sahi method override karo.
500 Internal Server Error	Servlet me exception. `logs\localhost.*.log` ya console (`catalina.bat run`) me stack trace dekho. NullPointerException / SQLException aam hain.
ClassNotFoundException: javax.servlet...	Tomcat 10+ par **javax** import. **jakarta** karo.
javac: package jakarta.servlet does not exist	Compile me `-cp C:\tomcat\lib\servlet-api.jar` nahi diya.
Port 8080 already in use	`netstat -ano | findstr :8080` → `taskkill /PID … /F` , ya server.xml me port badlo.
JAVA_HOME not defined correctly	Variable JDK folder par point kare (bin par nahi). Naya CMD kholo.
Tomcat window turant band	`catalina.bat run` se chalao — error usi window me dikhega.
Change dikh nahi raha	Class recompile kiya to Tomcat restart; browser cache clear (Ctrl+F5).
No suitable driver	mysql-connector jar sahi lib folder me nahi / URL galat.
Access denied for user 'root'	Password galat — DBUtil.java / context.xml me sahi password.
Communications link failure	MySQL service band (services.msc → MySQL80 start), port 3306 check.
Hindi / special text ?? dikhe	EncodingFilter + `charset=UTF-8` + HTML me `<meta charset="UTF-8">` .
IllegalStateException (forward)	forward/redirect ke baad dobara response likha — `return;` lagao.
Login ke baad bhi login page	Session me user set nahi hua ya cookie block. Login ke baad redirect ho raha hai? AuthFilter pattern check.

Quick comparison sheet

Servlet & Tomcat

Topic	Remember
Layout	WEB-INF\{web.xml, classes, lib} · META-INF\context.xml · public files root me
Lifecycle	init (1×) → service/doGet/doPost (har request) → destroy (1×)
init-param / context-param	ServletConfig (ek servlet) / ServletContext (poori app)
forward / include / redirect	1 request, URL same / merge output / 2 requests, URL change
Filter	init · doFilter(...chain.doFilter) · destroy; order = web.xml mapping order
Cookie / Session	browser me (≤4KB) / server me (JSESSIONID cookie)
Logout	getSession(false) → invalidate() → redirect /login

JDBC & Applet

Topic	Remember
JDBC steps	driver → Connection → Statement → execute → ResultSet → close
MySQL	`com.mysql.cj.jdbc.Driver` · `jdbc:mysql://localhost:3306/college`
execute methods	Query=SELECT/ResultSet · Update=DML/int · execute=any/boolean
Transaction	setAutoCommit(false) → commit() / rollback()
Jar location	WEB-INF\lib (DriverManager) · C:\tomcat\lib (pool)
Applet lifecycle	init → start → paint → stop → destroy
Applet run	AppletRunner (JDK 11+) / appletviewer (JDK 8)

Viva — short Q & A

Question	Answer
Servlet container kaun sa hai?	Apache Tomcat
Default Tomcat port?	8080 (HTTP), 8005 (shutdown)
CATALINA_HOME kya hai?	Tomcat ka install folder
Servlet ka parent class?	HttpServlet (→ GenericServlet)
doGet vs doPost?	GET: URL me data, bookmark; POST: body me data, secure/large
Servlet kitni baar init hota hai?	Ek baar
Session id kis cookie me?	JSESSIONID
Default session timeout?	30 minutes
JDBC driver type MySQL ka?	Type 4 (thin, pure Java)
PreparedStatement kyon?	SQL injection se bachav + faster
WAR ka full form?	Web Application Archive
Applet me main() hota hai?	Nahi; init() se shuru

Command cheat-sheet (Windows)

```bash
:: Tomcat
C:\tomcat\bin\startup.bat
C:\tomcat\bin\shutdown.bat
C:\tomcat\bin\catalina.bat run

:: Build + deploy project
cd C:\myapp-dev
build.bat

:: Console JDBC
javac StudentCrud.java
java -cp ".;mysql-connector-j-8.3.0.jar" StudentCrud

:: Applet
java AppletRunner HelloApplet

:: Port problem
netstat -ano | findstr :8080
taskkill /PID <pid> /F
```

Revision checklist

- ☐ Tomcat install: JDK, zip, JAVA_HOME, CATALINA_HOME, startup.bat, localhost:8080
- ☐ Tomcat folders aur web-app folder (WEB-INF) draw kar sakta hoon
- ☐ web.xml tags, context.xml DataSource, server.xml port
- ☐ Hello servlet + lifecycle + init-param vs context-param
- ☐ forward / include / redirect table aur code
- ☐ Filter: lifecycle, chain, mapping, Auth filter
- ☐ Cookie create/read/delete, Session API, login-logout code
- ☐ JDBC steps, Statement types, CRUD, transaction
- ☐ Servlet + MySQL (DAO) aur connection pool
- ☐ Applet lifecycle, param, event, paint, run method